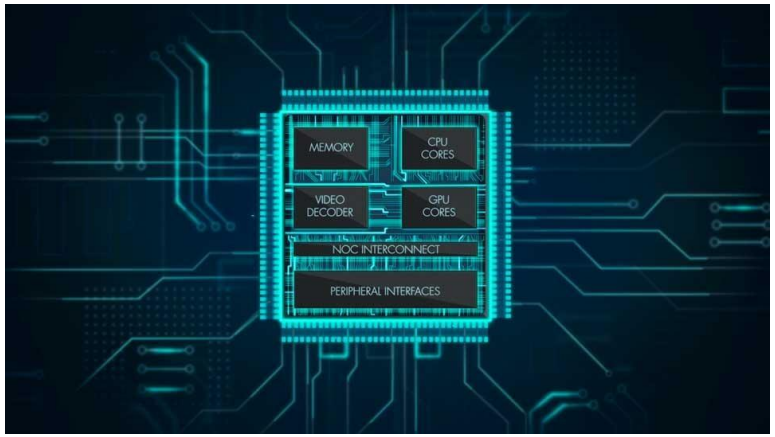
Digital System Design

Basic Practice 3



Date : 2026. 04. 09.

Instructor : **Prof. Hyun Kim**

Lecture Note : eclass.seoultech.ac.kr

Office : Mirae Hall #526

Web : idsl.seoultech.ac.kr

Contact : hyunkim@seoultech.ac.kr / 010-9600-5427

Contents

- Ultra Random Access Memory (URAM)
- Problem 1: URAM Write And Read
- GEMV Operation
- Problem 2: GEMV Operation with URAM
- Problem 3: GEMV Operation with LUTRAM
- Appendix: URAM Tiling and Usage

Contents

- Ultra Random Access Memory (URAM)
- Problem 1: URAM Write And Read
- GEMV Operation
- Problem 2: GEMV Operation with URAM
- Problem 3: GEMV Operation with LUTRAM
- Appendix: URAM Tiling and Usage

Ultra Random Access Memory

UltraRAM (URAM)

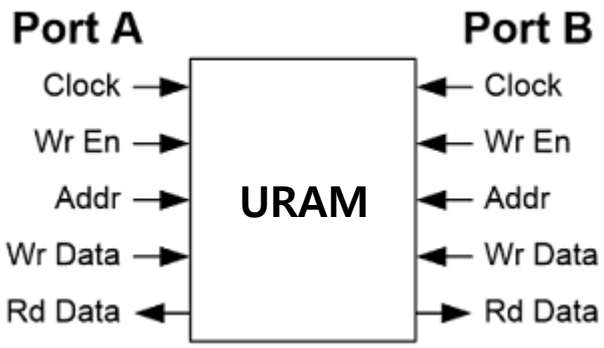
- UltraRAM is a memory in FPGA fabric for **on-chip storage.**
- Available KV260 UltraRAM Resources.

Table 5: **PL Resources**

Resource	K26 SOM	Description
System logic cells	256,200	Programmable logic cells for available
CLB flip-flops	234,240	Configurable logic block (CLB): Total number of flip-flops
CLB LUTs	117,120	Configurable logic block: Total number of look-up tables
Distributed RAM (Mb)	3.5	Distributed memory
Block RAM	144	Number of 36 Kb block RAMs
Block RAM (Mb)	5.1	Total block RAM memory footprint
UltraRAM blocks	64	288 Kb dual-port, 72-bit-wide memory with error correction
DSP slices	1,248	27 x 18 signed multiplier with 48-bit adder/accumulator
GTH transceivers	4	Up to 12.5 Gb/s serial transceivers
Video Codec	1	H.264 and H.265 supported simultaneous encode/decode
HDIO	69	High-density I/O supports 1.2V to 3.3V rails
HPIO	116	High-performance I/O differential pairs supports 1.0V to 1.8V rails

How to use a URAM (1)

- **URAM supports dual-port mode only**
  - It comes in up to **288k bits**, with availability depending on the device.
  - UltraRAM provides **two independent ports**, enabling **parallel memory access in the same clock cycle**.
- **Compared against BRAM**
  - URAM is a **high-density memory** for large/deep buffers, whereas BRAM is a **more flexible memory** for smaller and irregular buffering needs.
- **Only one way to implement URAM in Vivado**
  - The way is to **write a URAM inferring code** so that the compiler can recognize that the code is for URAM and implement it as URAM.



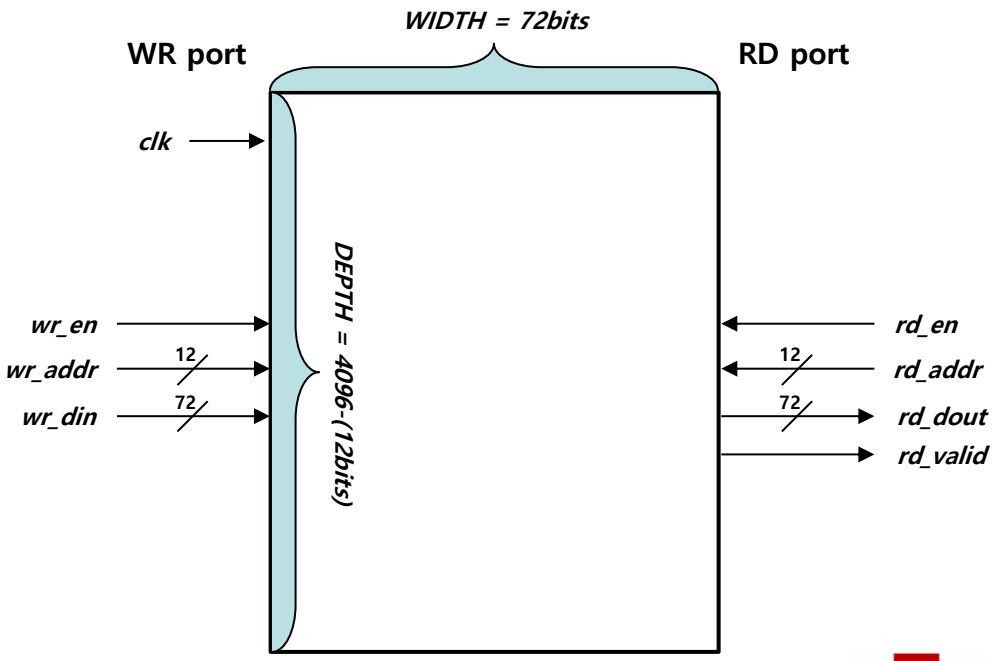
< Dual Port URAM >

How to use a URAM (2)

Simple Dual Port URAM inferring code

```
module simple_dual_port_uram #(
    parameter WIDTH      = 72,
    parameter DEPTH      = 4096,
    parameter ADDR_WIDTH = $clog2(DEPTH),
    parameter INIT_FILE  = ""
)(
    input  wire                clk,
    input  wire                wr_en,
    input  wire                rd_en,
    input  wire [ADDR_WIDTH-1:0] wr_addr,
    input  wire [ADDR_WIDTH-1:0] rd_addr,
    input  wire [WIDTH-1:0]      wr_din,
    output reg                   rd_valid,
    output reg  [WIDTH-1:0]      rd_dout
);
    // URAM
    (* ram_style = "ultra" *) reg [WIDTH-1:0] mem [0:DEPTH-1];

 generate
        if (INIT_FILE != "") begin : use_init_file
            initial begin
                $readmemh(INIT_FILE, mem);
            end
        end else begin : init_to_zero
            integer i;
            initial begin
                for (i = 0; i < DEPTH; i = i + 1)
                    mem[i] = {WIDTH{1'b0}};
            end
        end
    endgenerate
```

```
    always @(posedge clk) begin
        if (wr_en)
            mem[wr_addr] <= wr_din;

        if (rd_en) begin
            rd_dout <= mem[rd_addr]; // READ_FIRST
        end

        rd_valid <= rd_en;
    end

endmodule
```



< Simple Dual Port URAM >

Contents

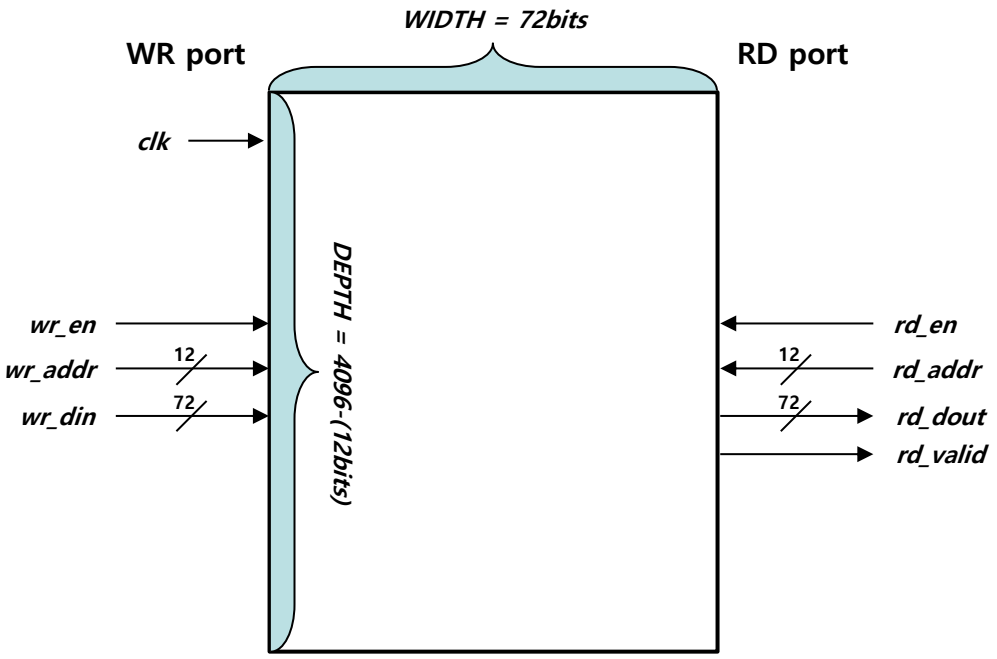
- Ultra Random Access Memory (URAM)
- Problem 1: URAM Write And Read
- GEMV Operation
- Problem 2: GEMV Operation with URAM
- Problem 3: GEMV Operation with LUTRAM
- Appendix: URAM Tiling and Usage

Prob. 1: URAM Write And Read

URAM write/read operation

- Write four 128-bit values to addresses 0 to 3 and read back the stored values.
- Verify that **"rd_dout"** matches the written data when **"rd_valid"** is asserted.

```
// Instantiation

simple_dual_port_uram #(
    .WIDTH(128),
    .DEPTH(4),
    .ADDR_WIDTH(),
)uram_inst(
    .clk(),
    .wr_en(),
    .rd_en(),
    .wr_addr(),
    .rd_addr(),
    .wr_din(),

    .rd_valid(),
    .rd_dout()
);
```



< Simple Dual Port URAM >

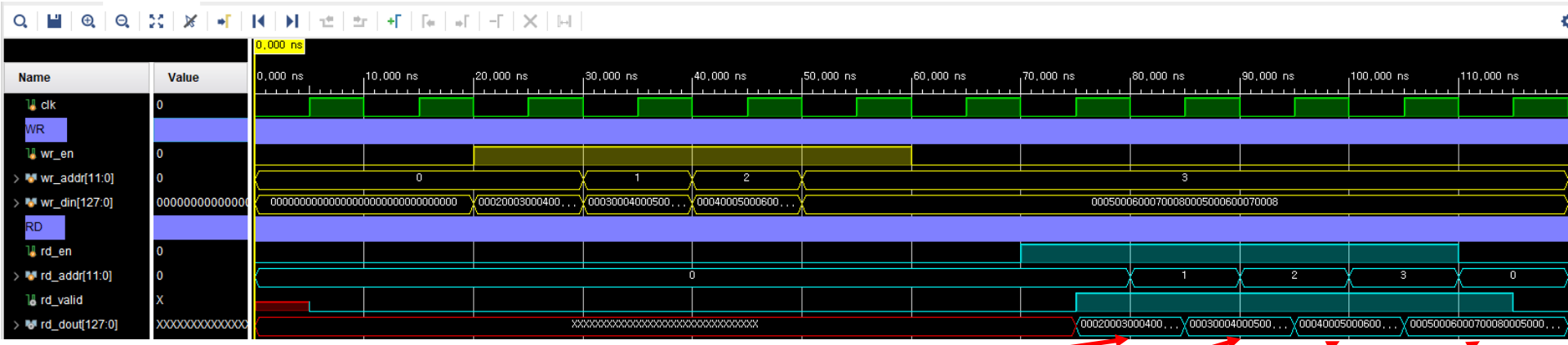
Prob. 1: Testbench

Testbench

```
`timescale 1ns / 1ps

module tb_prob1();
    parameter WIDTH      = 128;
    parameter DEPTH      = 4096;
    parameter ADDR_WIDTH = $clog2(DEPTH);

    reg                    clk;
    reg                    wr_en;
    reg                    rd_en;
    reg  [ADDR_WIDTH-1:0]  wr_addr;
    reg  [ADDR_WIDTH-1:0]  rd_addr;
    reg  [WIDTH-1:0]       wr_din;

    wire                   rd_valid;
    wire [WIDTH-1:0]       rd_dout;

    simple_dual_port_uram #(
        .WIDTH(WIDTH),
        .DEPTH(DEPTH),
        .ADDR_WIDTH(ADDR_WIDTH),
        .INIT_FILE("")
    ) uram_inst (
        .clk(clk),
        .wr_en(wr_en),
        .rd_en(rd_en),
        .wr_addr(wr_addr),
        .rd_addr(rd_addr),
        .wr_din(wr_din),
        .rd_valid(rd_valid),
        .rd_dout(rd_dout)
    );

    initial begin
        clk = 1'b0;
        forever #5 clk = ~clk;
    end

    initial begin
        wr_en   = 1'b0;
        rd_en   = 1'b0;
        wr_addr = 0;
        rd_addr = 0;
        wr_din  = 0;
        #20;
        wr_en   = 1'b1;
        wr_addr = 2'd0;
        wr_din  = 128'h0002_0003_0004_0005_0002_0003_0004_0005;
        #10;
        wr_addr = 2'd1;
        wr_din  = 128'h0003_0004_0005_0006_0003_0004_0005_0006;
        #10;
        wr_addr = 2'd2;
        wr_din  = 128'h0004_0005_0006_0007_0004_0005_0006_0007;
        #10;
        wr_addr = 2'd3;
        wr_din  = 128'h0005_0006_0007_0008_0005_0006_0007_0008;
        #10;
        wr_en   = 1'b0;
        #10;
        rd_en   = 1'b1;
        rd_addr = 2'd0;
        #10;
        rd_addr = 2'd1;
        #10;
        rd_addr = 2'd2;
        #10;
        rd_addr = 2'd3;
        #10;
        rd_en   = 1'b0;
        rd_addr = 0;
        #10;
        $finish;
    end
endmodule
```

Prob. 1: Waveform

Waveform



00020003000400050002000300040005

00030004000500060003000400050006

00040005000600070004000500060007

00050006000700080005000600070008

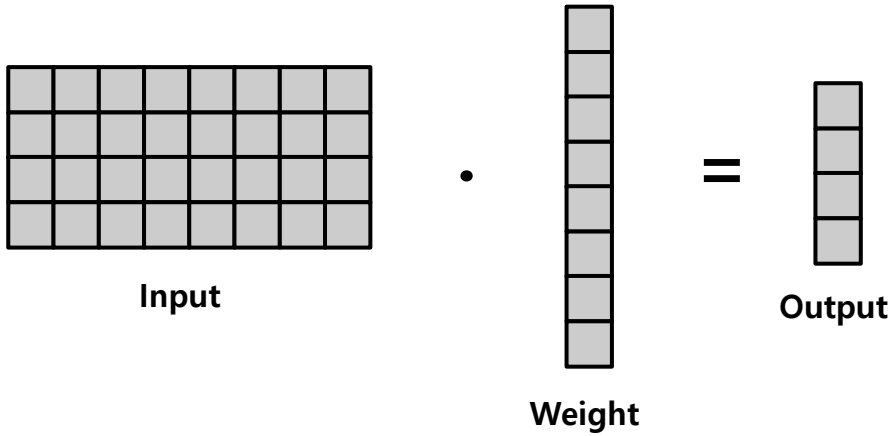
Contents

- Ultra Random Access Memory (URAM)
- Problem 1: URAM Write And Read
- GEMV Operation
- Problem 2: GEMV Operation with URAM
- Problem 3: GEMV Operation with LUTRAM
- Appendix: URAM Tiling and Usage

GEMV Operation (1)

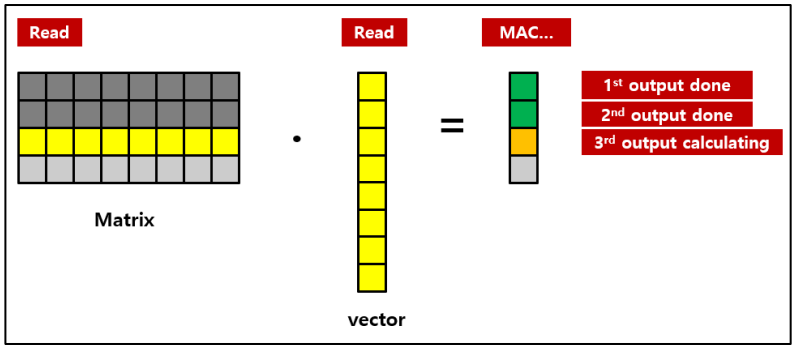
- **GEneral Matrix-Vector multiplication (GEMV)**
  - Multiplies a matrix and a vector, resulting in a vector.
- **Weight Stationary**
  - The **same weight vector is kept** for successive row computations, while input rows are streamed in one by one. As a result, the design **reuses the fixed weights** across multiple rows and generates the outputs sequentially.
- **Output Stationary**
  - The output values stay in local storage during the computation and are **updated step by step as new data arrives**. After all contributions have been accumulated, the completed outputs are **written back to the output memory**.

$$y = Wx$$

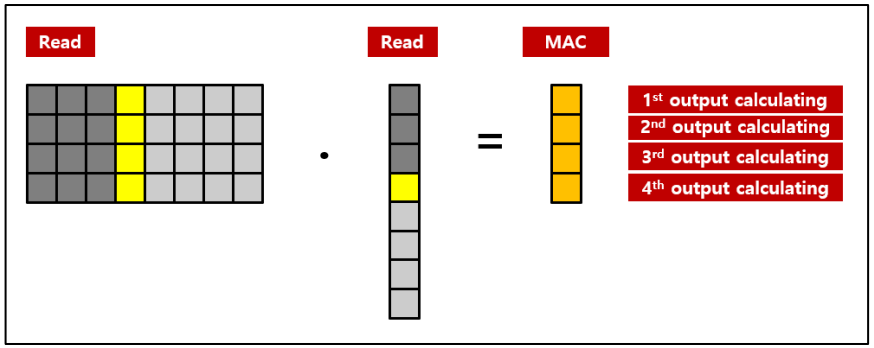


GEMV Operation (2)

- **The biggest difference between the two methods is how often you have to access the data.**
  - In the first case, the outputs can be generated row by row with a relatively **simple computation flow**. however, the same data must be **accessed several times**.
  - In the second case, the **outputs remain stationary** in local storage and are **accumulated step by step**. This **reduces output data movement**, but requires **multi-step partial-sum accumulation**.



< Weight Stationary >



< Output Stationary >

- **It can directly impact performance, efficiency, and overall hardware architecture**

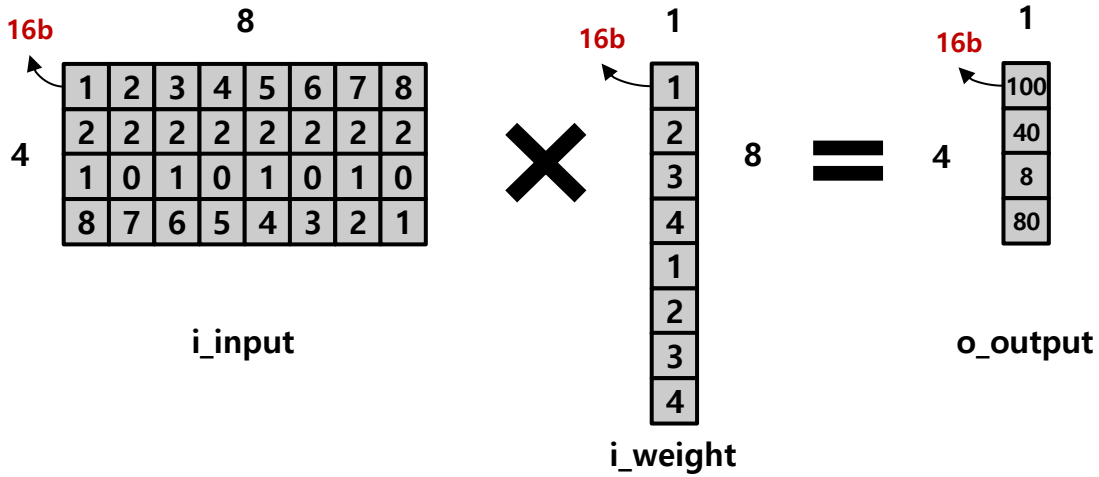
Contents

- Ultra Random Access Memory (URAM)
- Problem 1: URAM Write And Read
- GEMV Operation
- Problem 2: GEMV Operation with URAM
- Problem 3: GEMV Operation with LUTRAM
- Appendix: URAM Tiling and Usage

Prob. 2: GEMV Operation with URAM

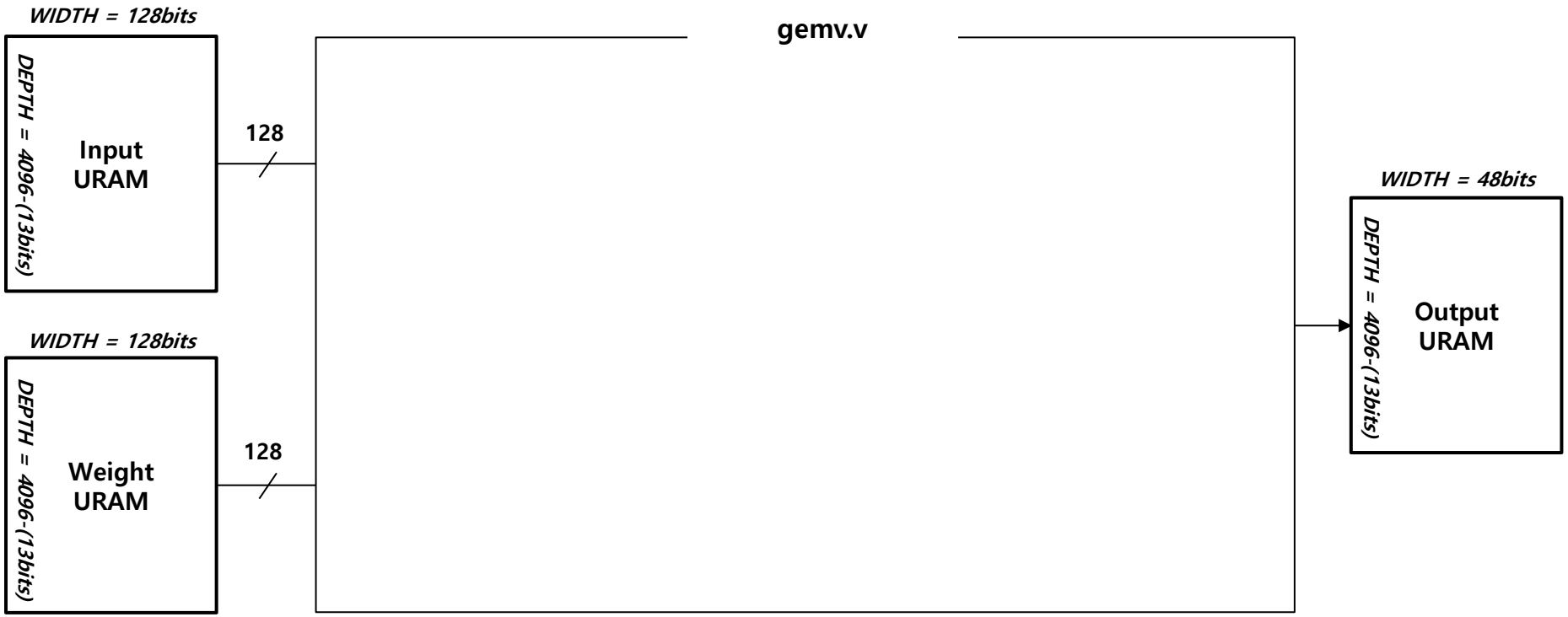
Design GEMV operation with DSPs, Adder-tree, URAMs

- Each input and weight word is a **128-bit packed** value containing eight 16-bit integers, and each output element is accumulated as a 48-bit integer.
- For one matrix row, eight 16-bit multiplications are executed **in parallel** and then reduced by an adder tree.
- The output vector elements are generated row by row and **written sequentially to the output URAM**.



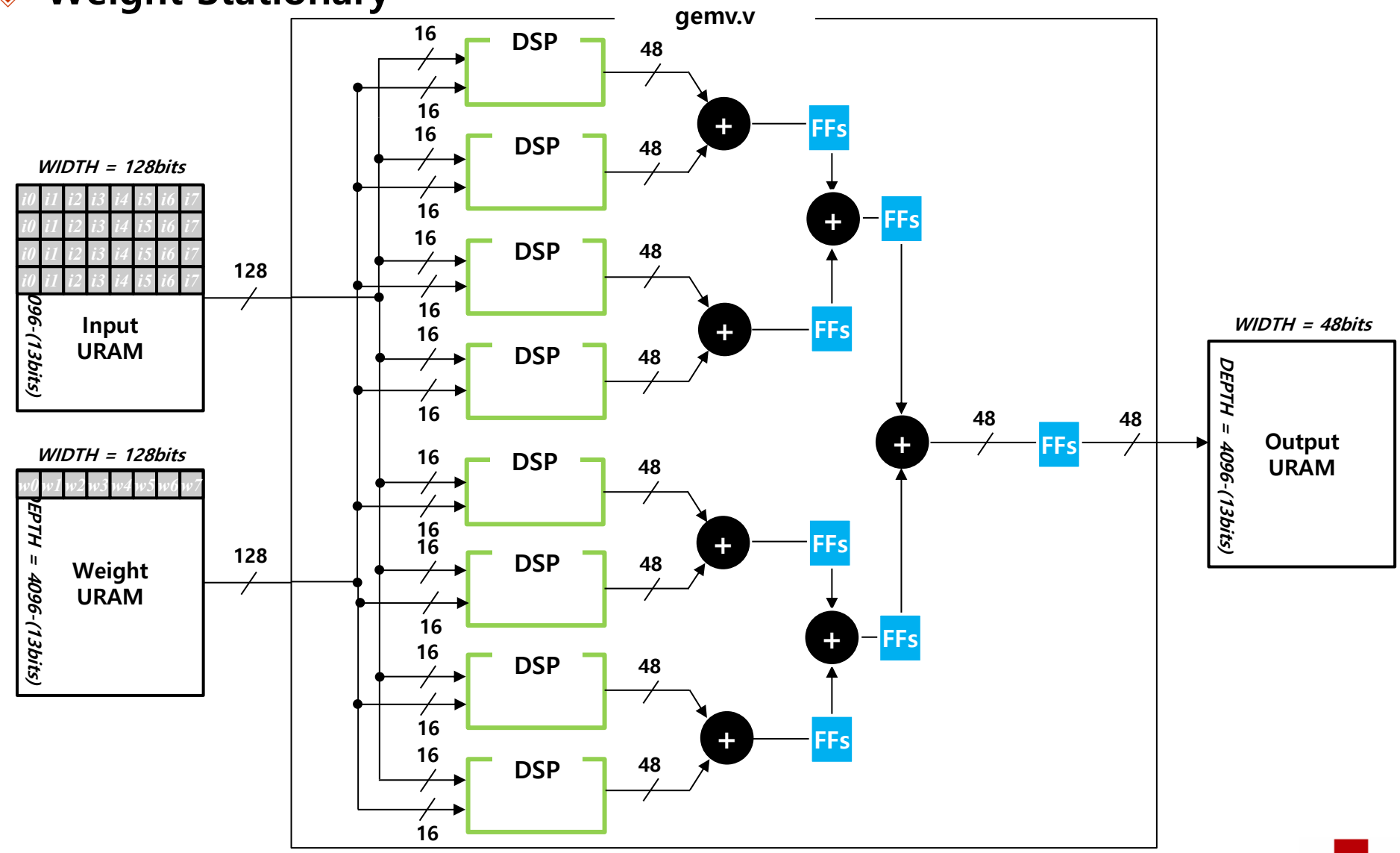
Prob. 2: GEMV Operation with URAM

- **Design a URAM-based vector multiplication module that reads multiple input and weight data and writes the result.**
  - Reuse the previously designed **128-bit simple dual-port URAM module** as-is.
  - Read **two 128-bit input and weight values per cycle**.
  - Perform **eight 16-bit multiplications in parallel** and reduce them through a **3-stage pipelined adder tree**.
  - Write **one 48-bit output element per cycle** to the **output URAM**.



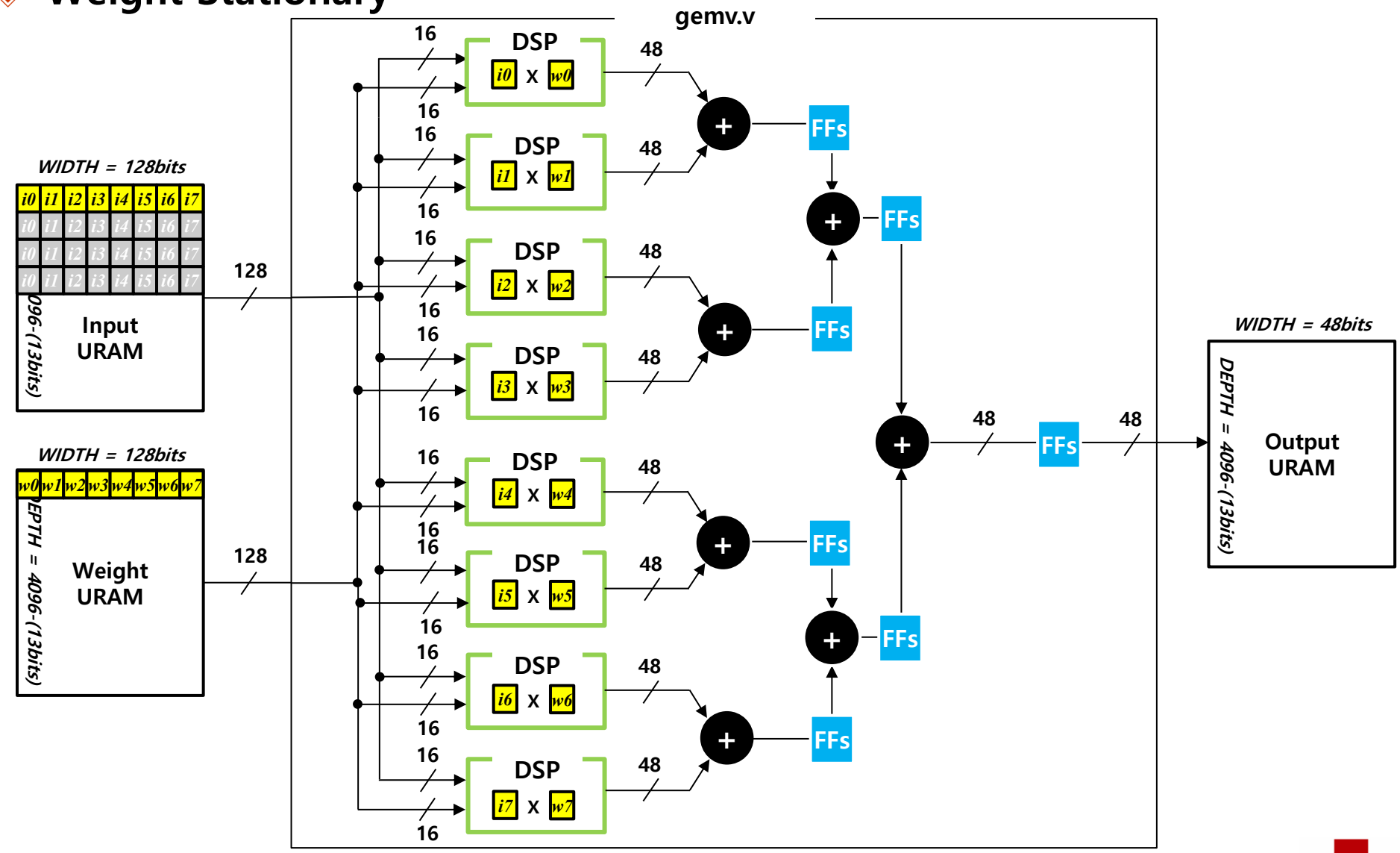
Prob. 2: GEMV Operation with URAM

Weight Stationary



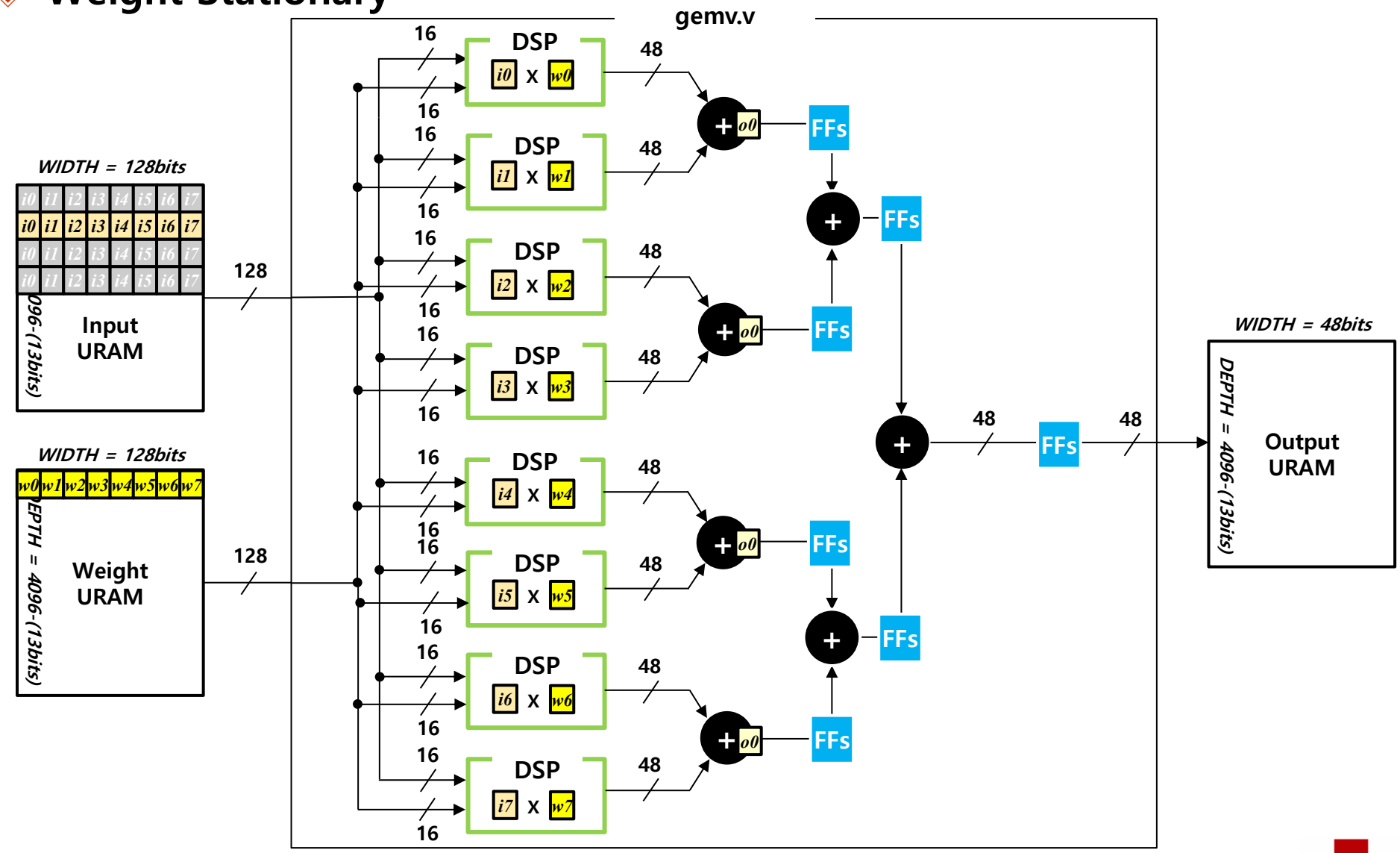
Prob. 2: GEMV Operation with URAM

Weight Stationary



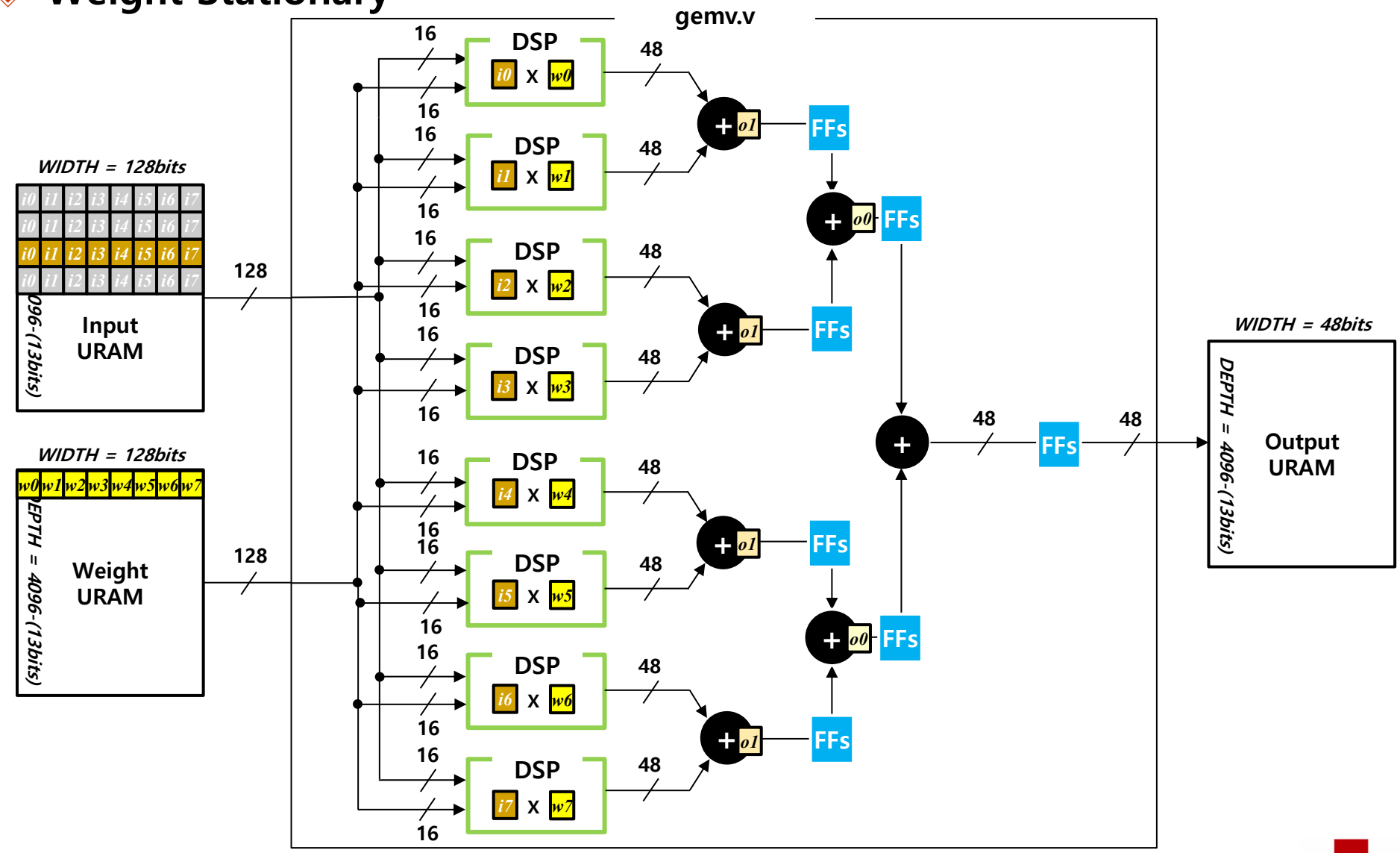
Prob. 2: GEMV Operation with URAM

Weight Stationary



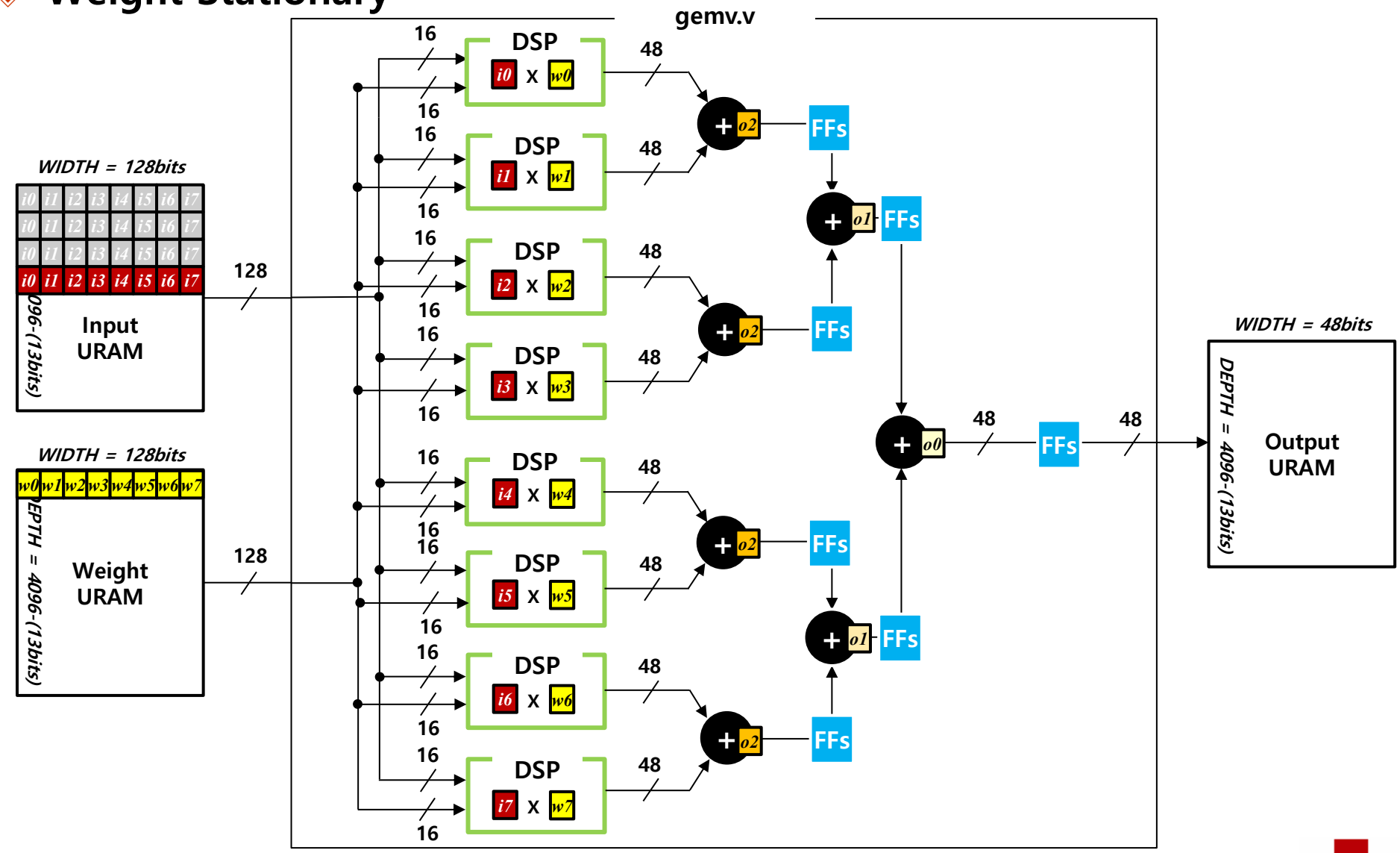
Prob. 2: GEMV Operation with URAM

Weight Stationary



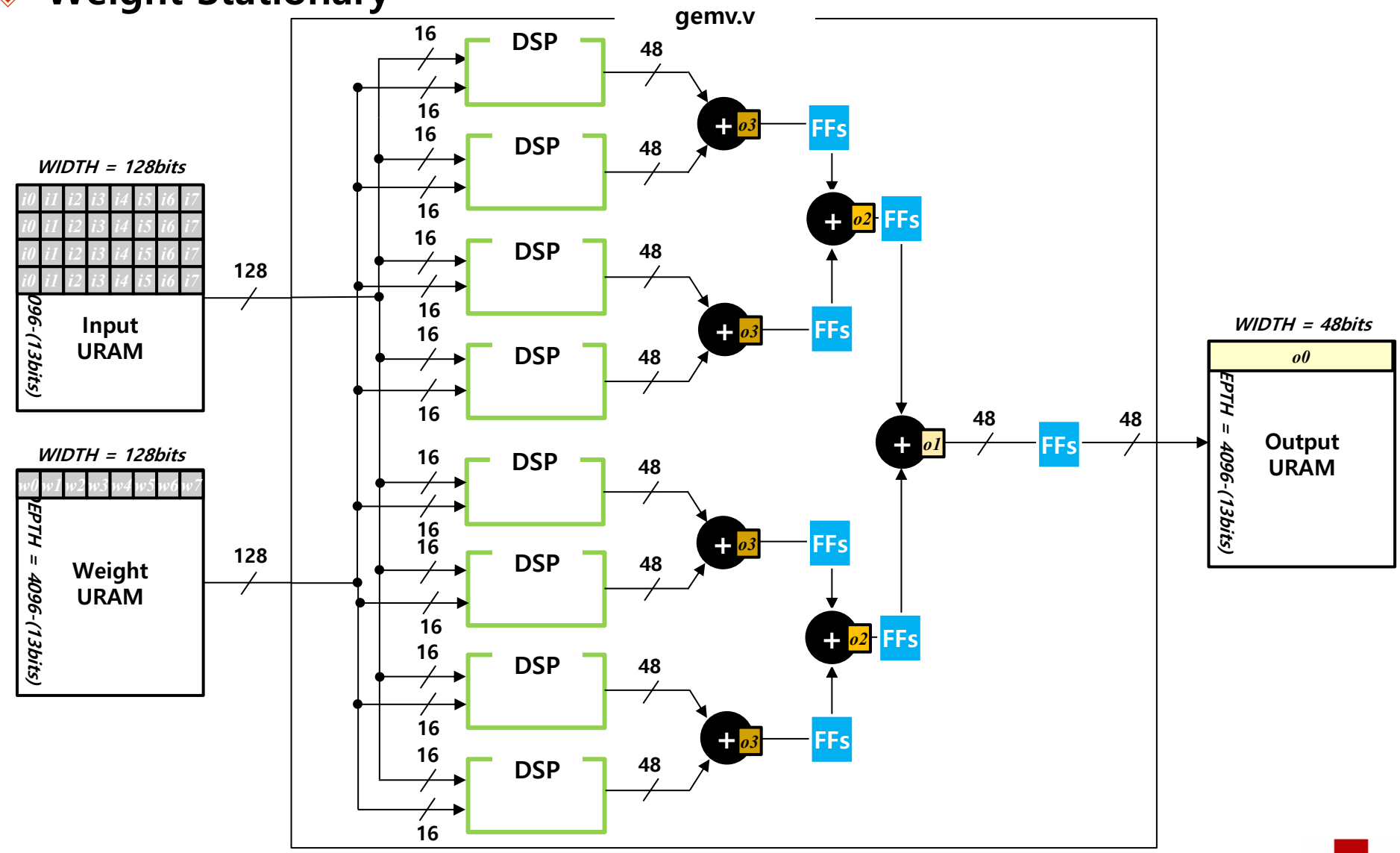
Prob. 2: GEMV Operation with URAM

Weight Stationary



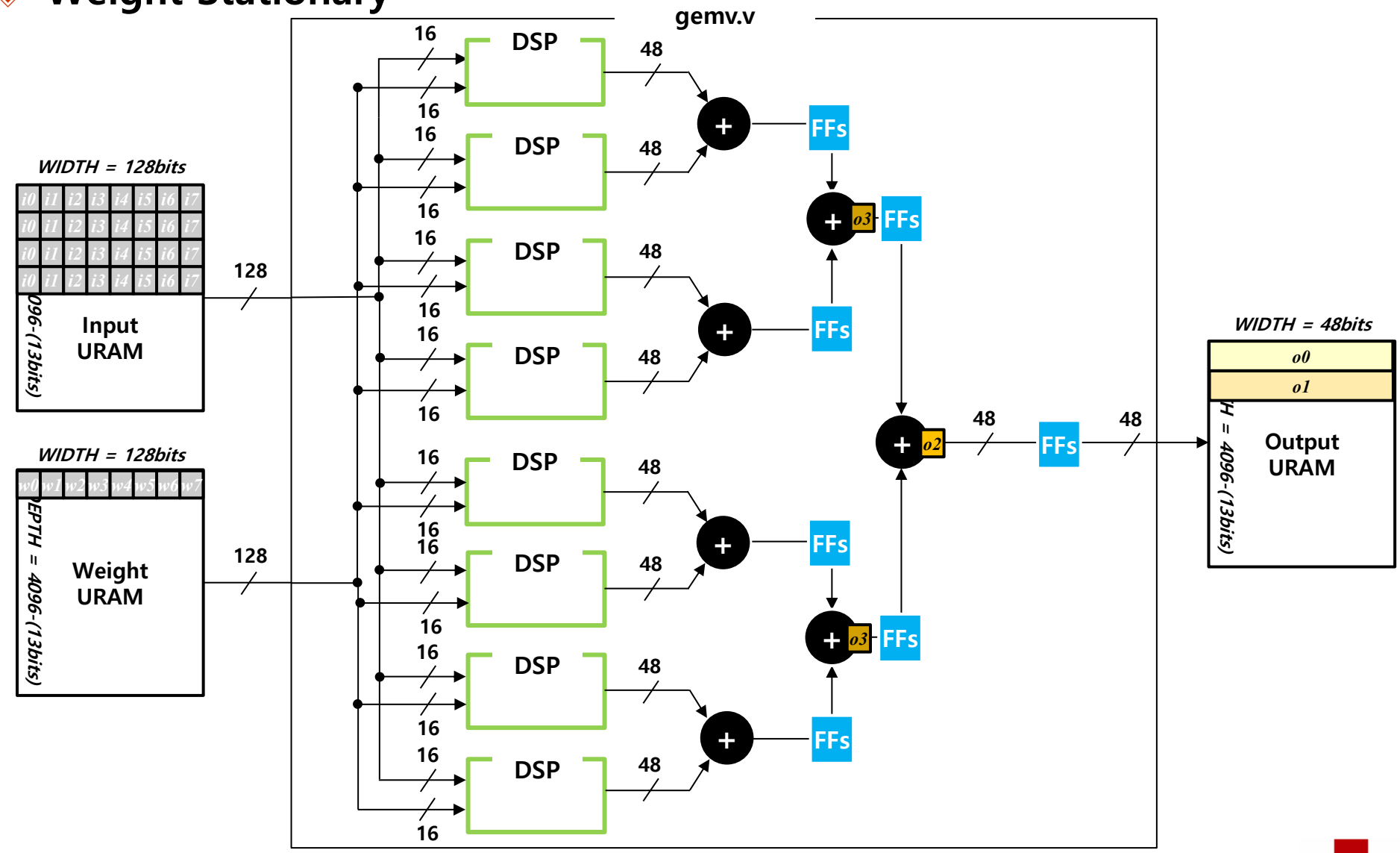
Prob. 2: GEMV Operation with URAM

Weight Stationary



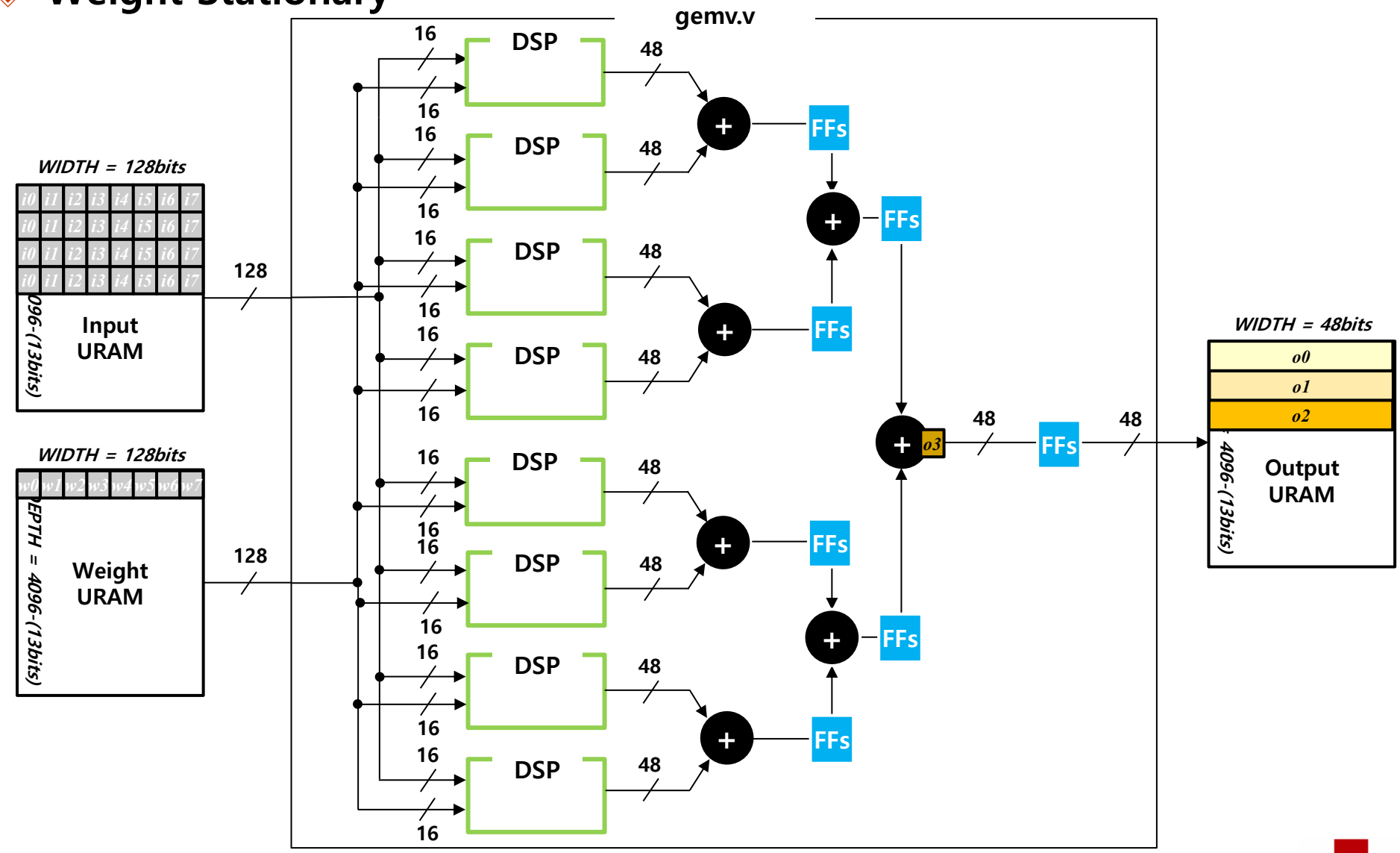
Prob. 2: GEMV Operation with URAM

Weight Stationary



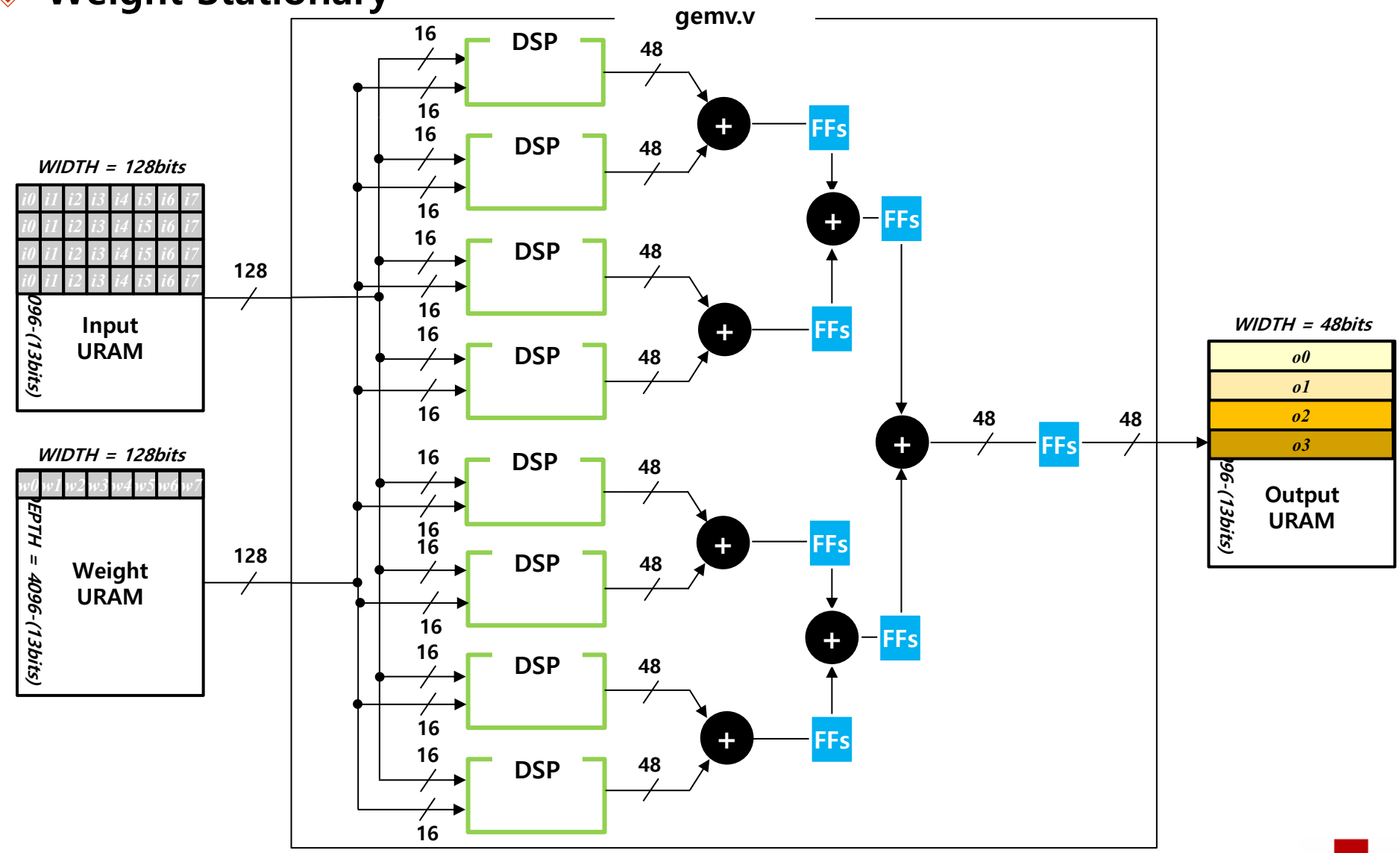
Prob. 2: GEMV Operation with URAM

Weight Stationary



Prob. 2: GEMV Operation with URAM

Weight Stationary



Prob. 2: GEMV Operation with URAM

Parameters for uram_based_gemv.v

```
parameter INPUT_WIDTH       =,
parameter WEIGHT_WIDTH      =,
parameter OUTPUT_WIDTH      =,

parameter INPUT_DEPTH       =,
parameter WEIGHT_DEPTH      =,
parameter OUTPUT_DEPTH      =,

parameter INPUT_ADDR_WIDTH  = $clog2(INPUT_DEPTH),
parameter WEIGHT_ADDR_WIDTH = $clog2(WEIGHT_DEPTH),
parameter OUTPUT_ADDR_WIDTH = $clog2(OUTPUT_DEPTH)
```

Prob. 2: GEMV Operation with LUTRAM

MAC.v

```
`timescale 1ns / 1ps

module MAC(
    input wire i_clk,
    input wire i_rstn,

    input wire i_dsp_enable,
    input wire signed [15:0] i_dsp_input,
    input wire signed [15:0] i_dsp_weight,

    output reg signed [47:0] o_dsp_output
    );

    wire signed [47:0] o_dsp_output_reg;

    always @(posedge i_clk) begin
        if (~i_rstn) begin
            o_dsp_output <= 48'sd0;
        end
        else if (i_dsp_enable) begin
            o_dsp_output <= o_dsp_output_reg;
        end
    end

    dsp_macro_0 dsp (
        .CLK(i_clk),  // input wire CLK
        .CE(i_dsp_enable),    // input wire CE
        .A(i_dsp_input),      // input wire [15 : 0] A
        .B(i_dsp_weight),      // input wire [15 : 0] B
        .P(o_dsp_output_reg)      // output wire [47 : 0] P
    );

endmodule
```

INSTANTIATION for MAC.v

```
MAC0 your_instance_name (
        .i_clk        (),
        .i_rstn       (),
        .i_dsp_enable(),
        .i_dsp_input (),
        .i_dsp_weight(),
        .o_dsp_output()
    );
```

Prob. 2: GEMV Operation with URAM

simple_line_lutram.v

```
`timescale 1ns / 1ps

module simple_line_lutram #(
    parameter WIDTH          = 128,
    parameter DEPTH          = 4,
    parameter ADDR_WIDTH     = $clog2(DEPTH),
    parameter RD_WIDTH       = 16,
    parameter BANK_DEPTH     = WIDTH / RD_WIDTH,
    parameter BANK_SEL_WIDTH = $clog2(BANK_DEPTH)
)(
    input  wire                    clk,
    input  wire                    wr_en,
    input  wire [ADDR_WIDTH-1:0]   wr_addr,
    input  wire [WIDTH-1:0]        wr_din,
    input  wire                    rd_en,
    input  wire [ADDR_WIDTH-1:0]   rd_addr,
    input  wire [BANK_SEL_WIDTH-1:0] rd_sel,
    output reg  [RD_WIDTH-1:0]     rd_dout
);

    wire [BANK_DEPTH*RD_WIDTH-1:0] bank_do;  // data from all banks
    wire [RD_WIDTH-1:0]            sel_do;   // selected bank output
```

```
genvar b;
    generate
        for (b = 0; b < BANK_DEPTH; b = b + 1) begin : gen_bank
            integer i;
            (* ram_style = "distributed" *) reg [RD_WIDTH-1:0] ram
[0:DEPTH-1]; // one LUTRAM bank

            initial begin
                for (i = 0; i < DEPTH; i = i + 1)
                    ram[i] = {RD_WIDTH{1'b0}};
            end

            always @(posedge clk) begin
                if (wr_en)
                    ram[wr_addr] <= wr_din[(b*RD_WIDTH) +: RD_WIDTH];
            end

            assign bank_do[(b*RD_WIDTH) +: RD_WIDTH] = ram[rd_addr];
        end
    endgenerate

    assign sel_do = bank_do[(rd_sel*RD_WIDTH) +: RD_WIDTH];

    always @(posedge clk) begin
        if (rd_en)
            rd_dout <= sel_do;
        else
            rd_dout <= {RD_WIDTH{1'b0}};
    end
endmodule
```

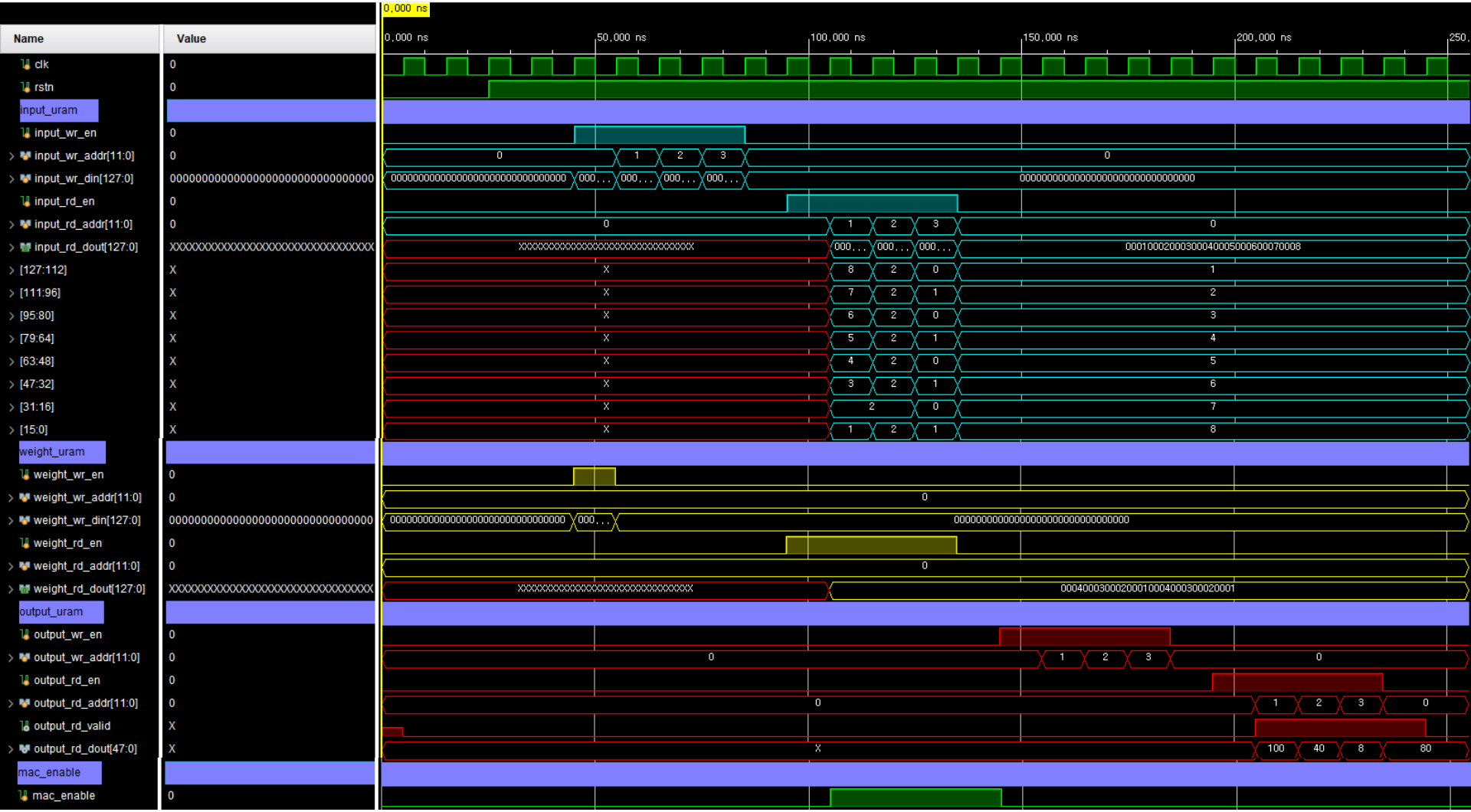
Prob. 2: GEMV Operation with URAM

INSTANTIATION for simple_line_lutram.v

```
simple_line_lutram #(
        .WIDTH          (),
        .DEPTH          (),
        .ADDR_WIDTH     (),
        .RD_WIDTH       (),
        .BANK_SEL_WIDTH()
    ) u_wt_lut (
        .clk     (),
        .wr_en   (),
        .wr_addr(),
        .wr_din (),
        .rd_en   (),
        .rd_addr(),
        .rd_sel (),
        .rd_dout()
    );
```

Prob. 2: Waveform

Waveform

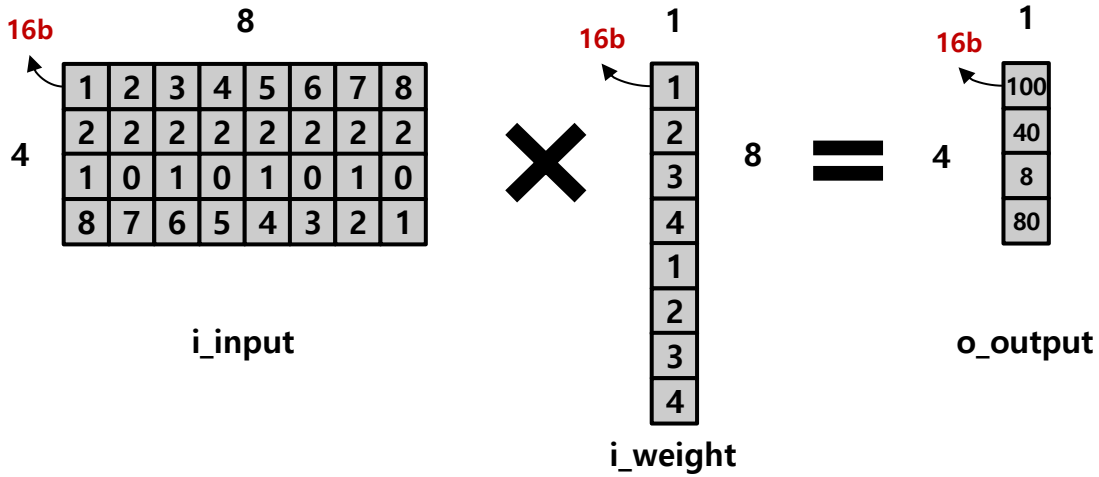


Contents

- Ultra Random Access Memory (URAM)
- Problem 1: URAM Write And Read
- GEMV Operation
- Problem 2: GEMV Operation with URAM
- Problem 3: GEMV Operation with LUTRAM
- Appendix: URAM Tiling and Usage

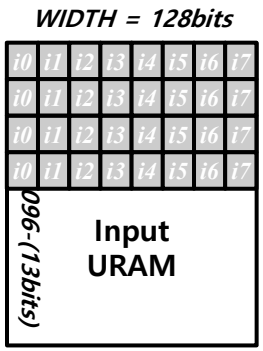
Prob. 3: GEMV Operation with LUTRAM

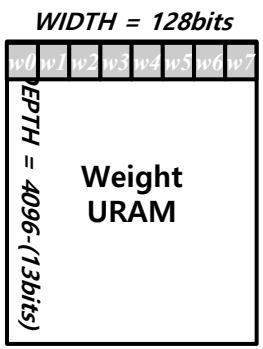
- **Design GEMV operation with DSPs, Adder-tree, URAMs, LUTRAMs**
  - Use LUTRAM as a local buffer between URAM and DSPs
  - Read **128-bit packed data** once from URAM, then access **16-bit elements flexibly from LUTRAM**.
  - Accumulate four outputs in parallel and store the final results in output URAM.

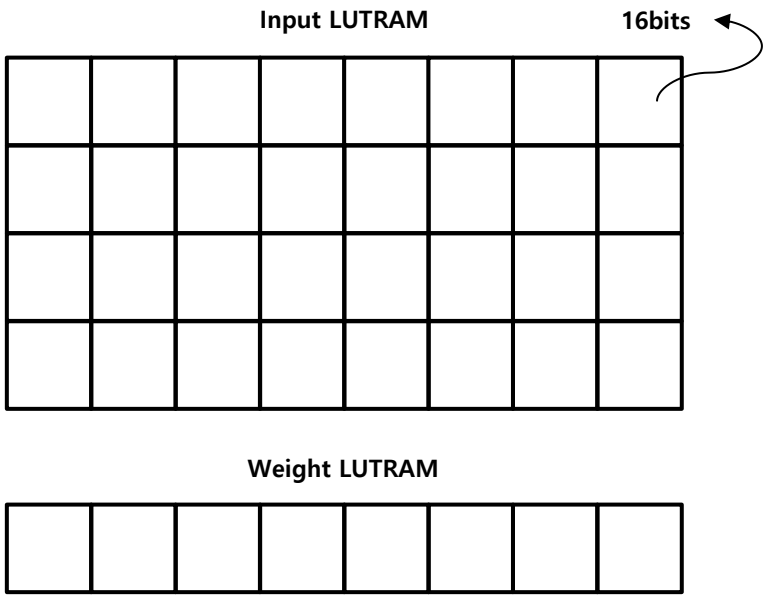


Prob. 3: GEMV Operation with LUTRAM

Output Stationary



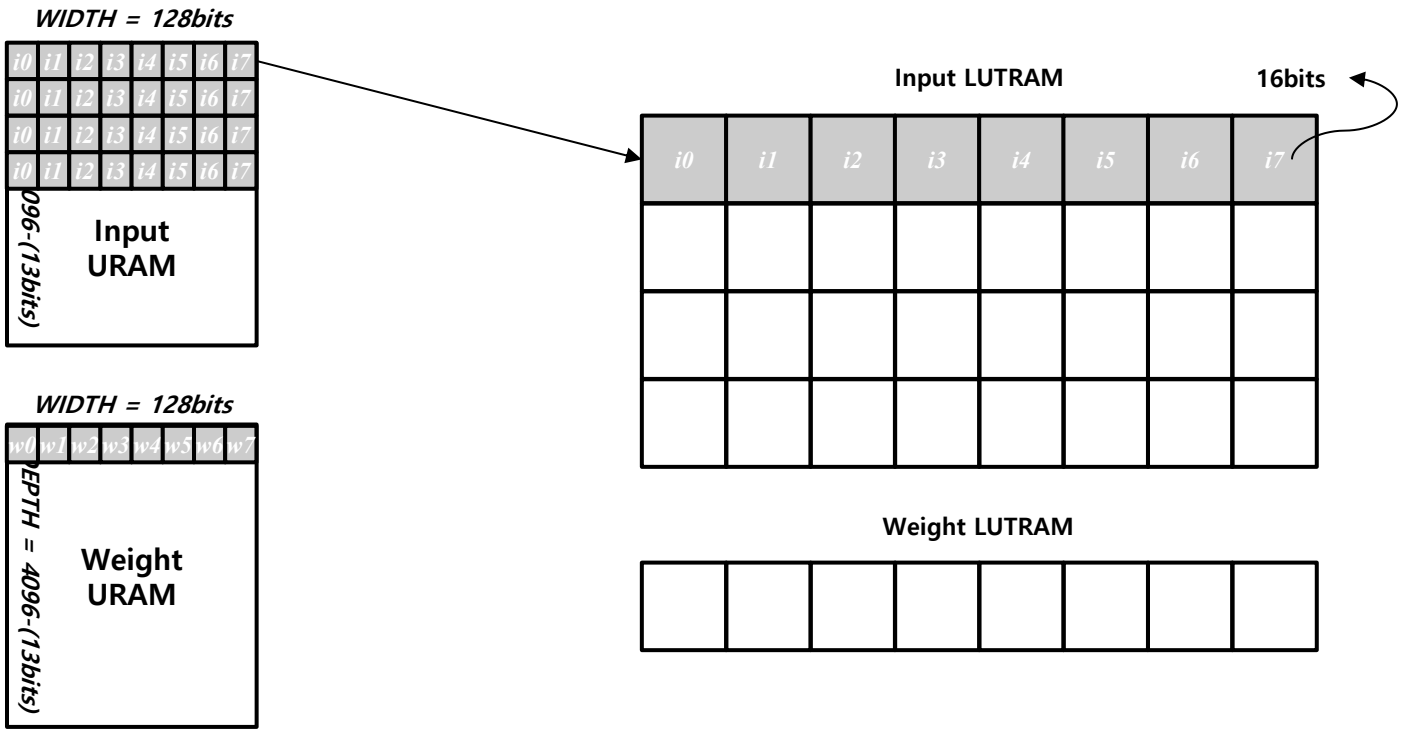




Prob. 3: GEMV Operation with LUTRAM

Output Stationary

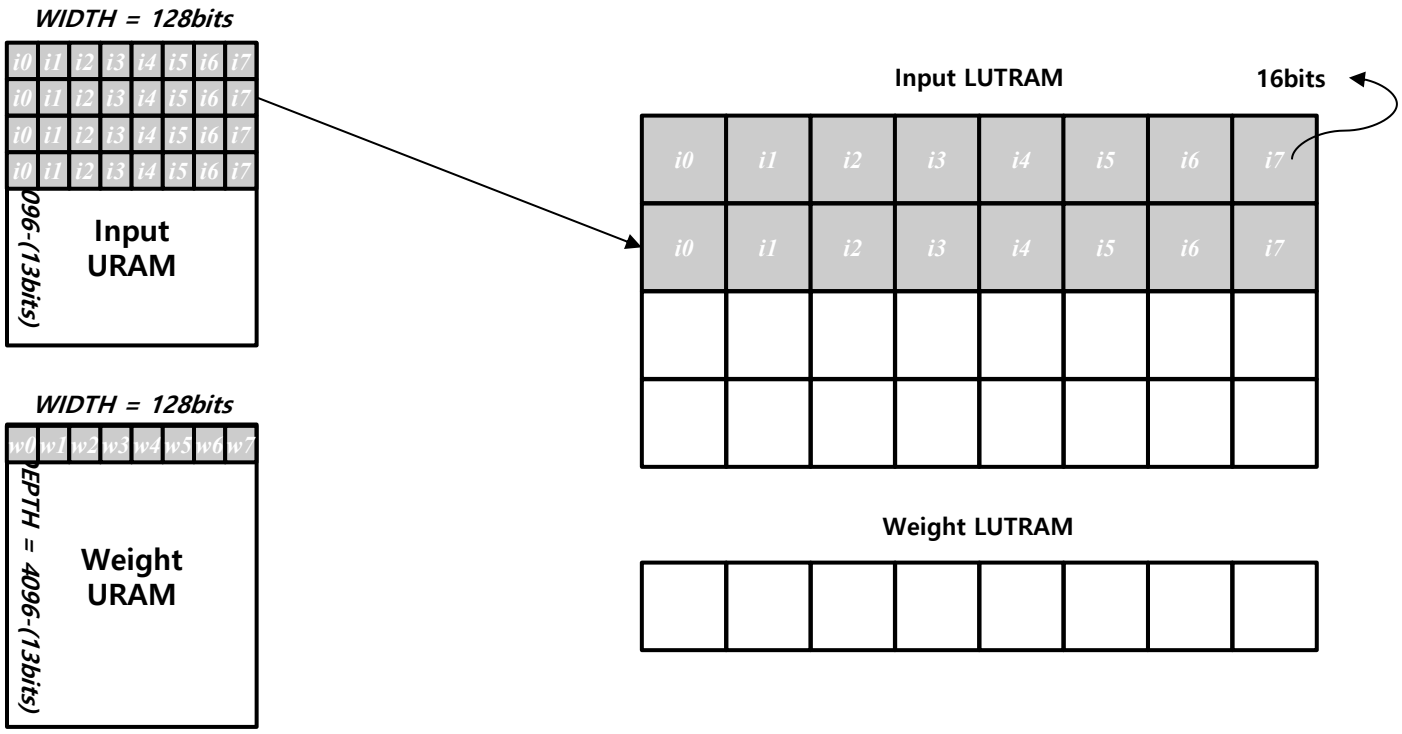
- Cycle 0



Prob. 3: GEMV Operation with LUTRAM

Output Stationary

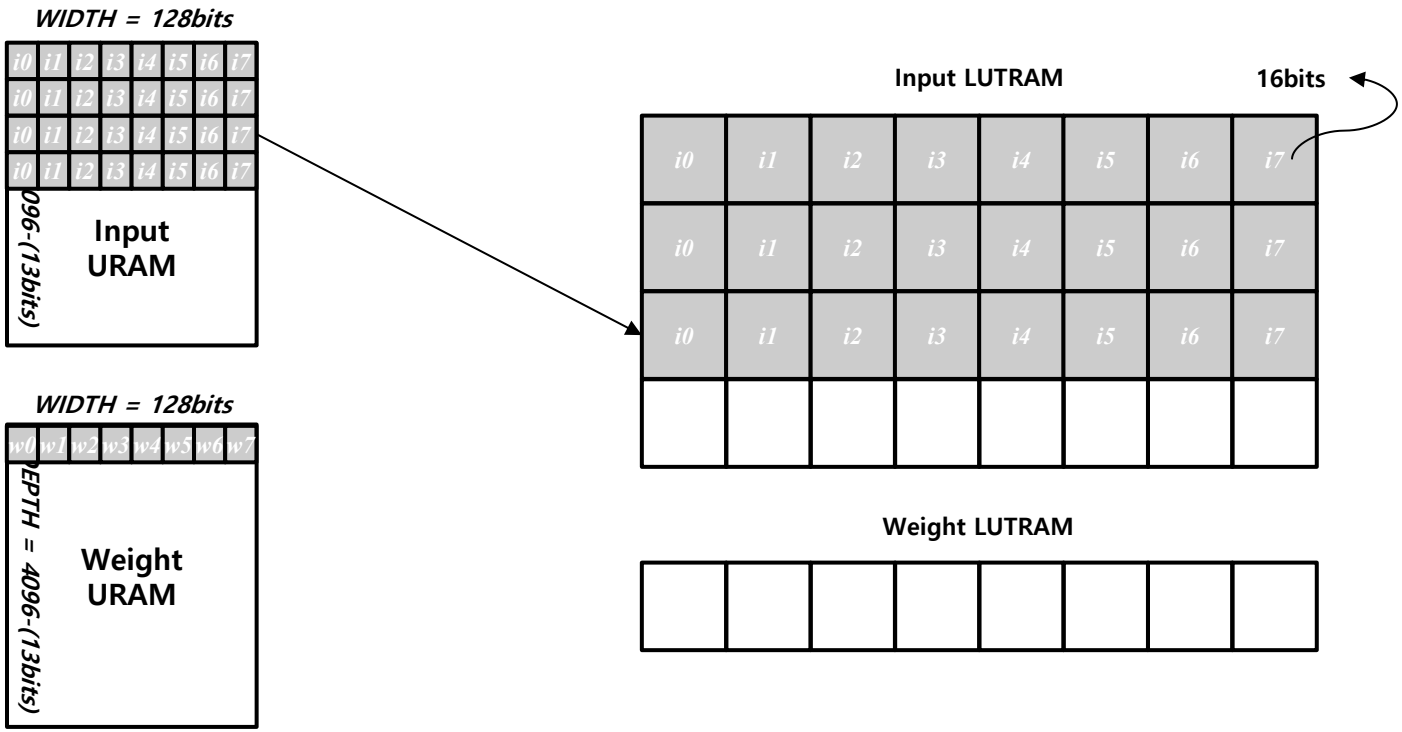
- Cycle 1



Prob. 3: GEMV Operation with LUTRAM

Output Stationary

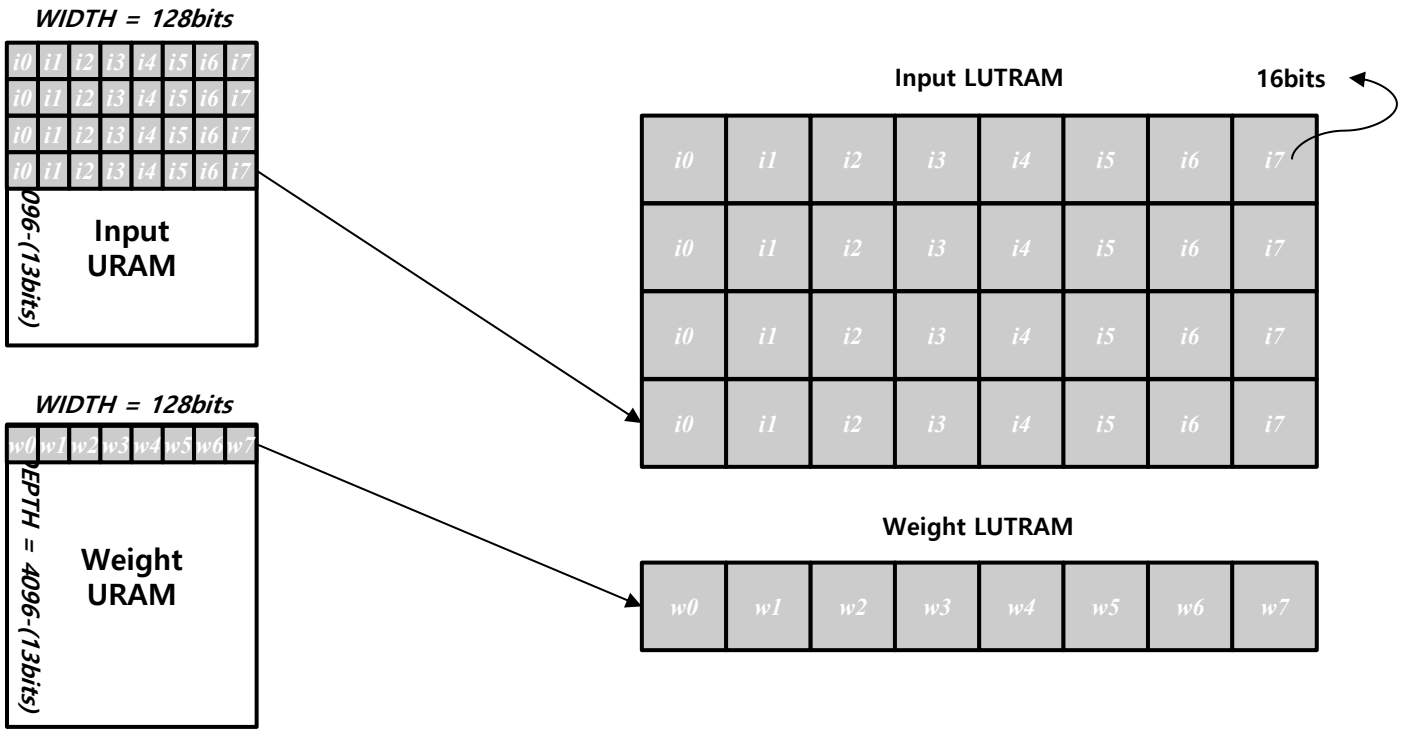
- Cycle 2



Prob. 3: GEMV Operation with LUTRAM

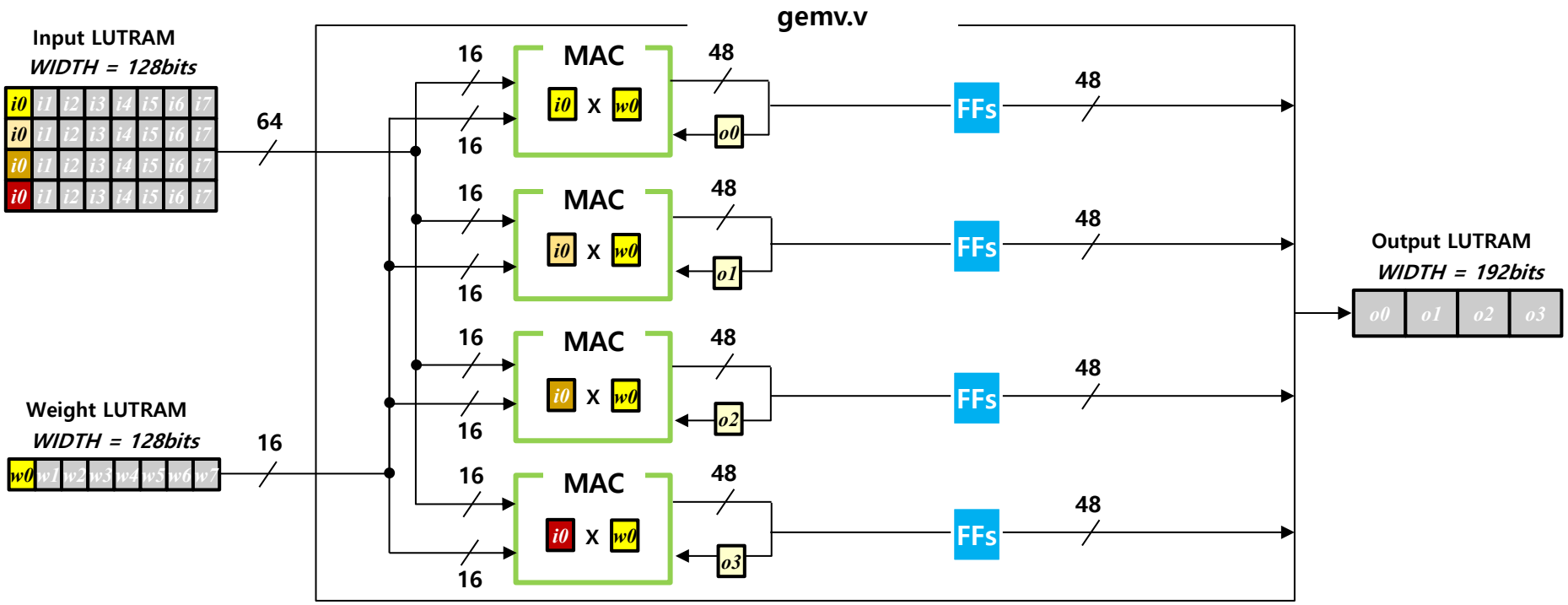
Output Stationary

- Cycle 3



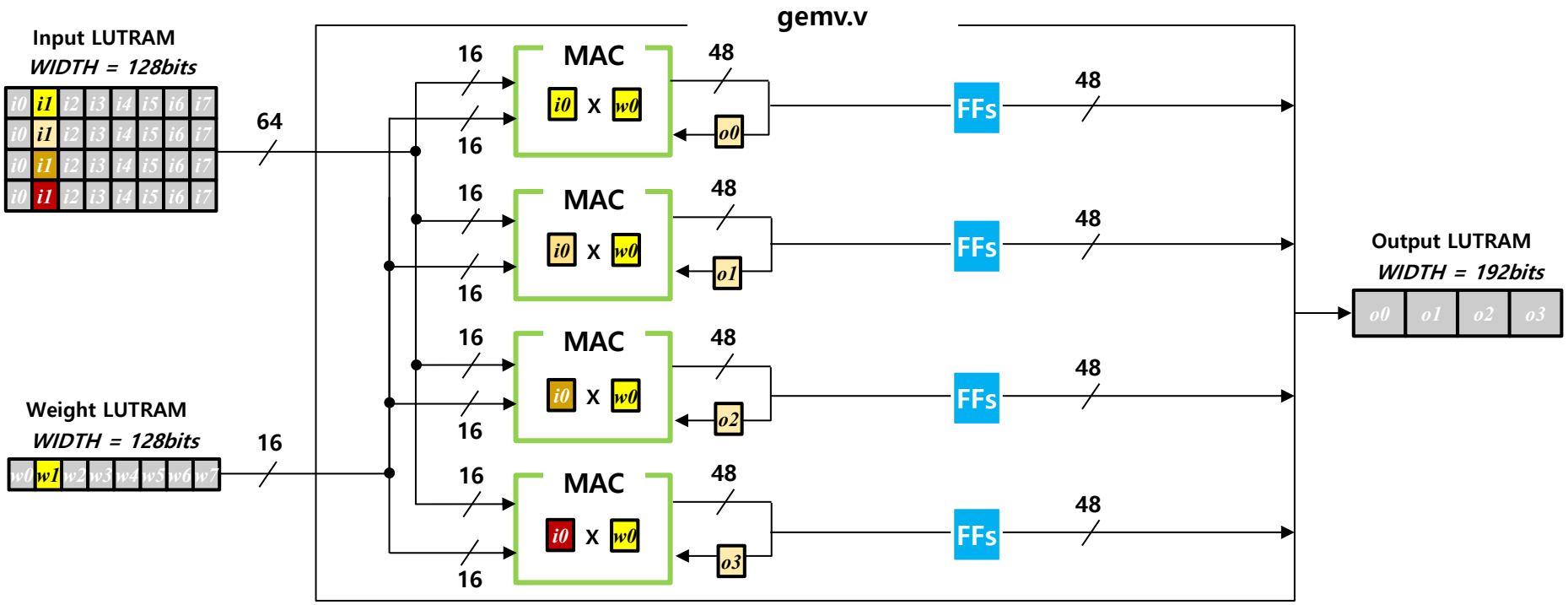
Prob. 3: GEMV Operation with LUTRAM

Output Stationary



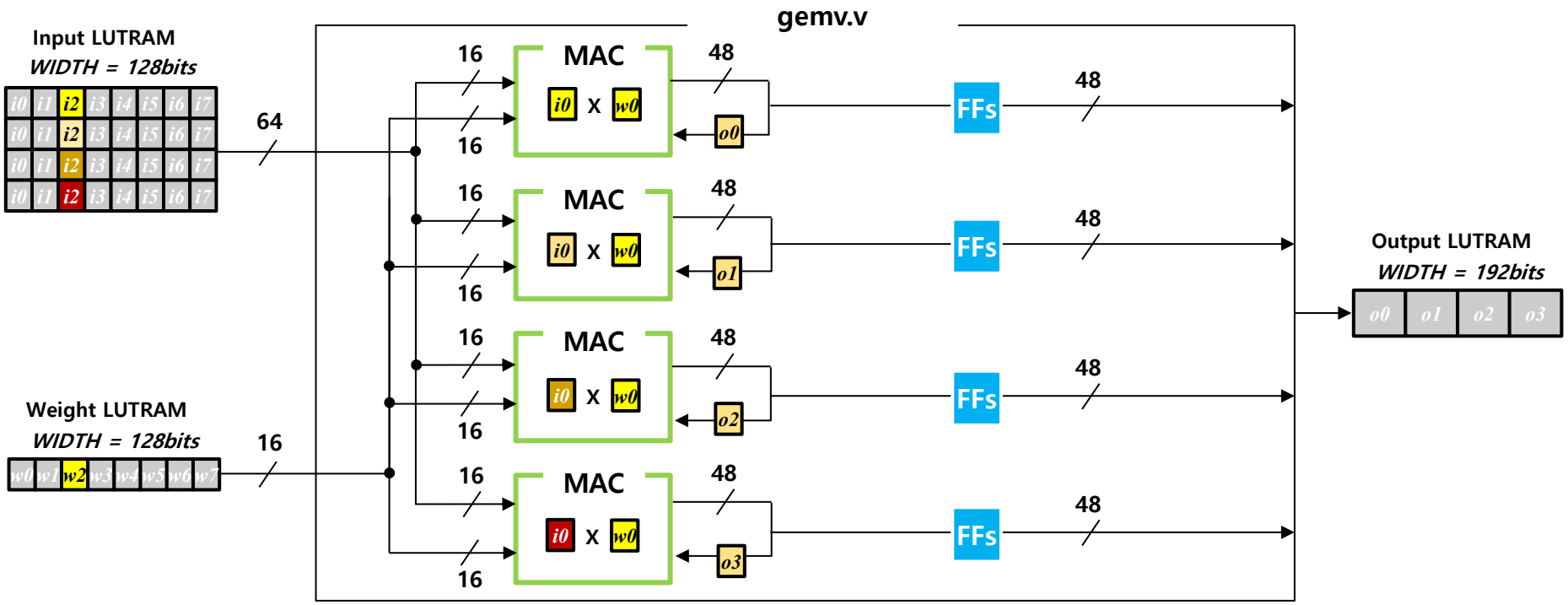
Prob. 3: GEMV Operation with LUTRAM

Output Stationary



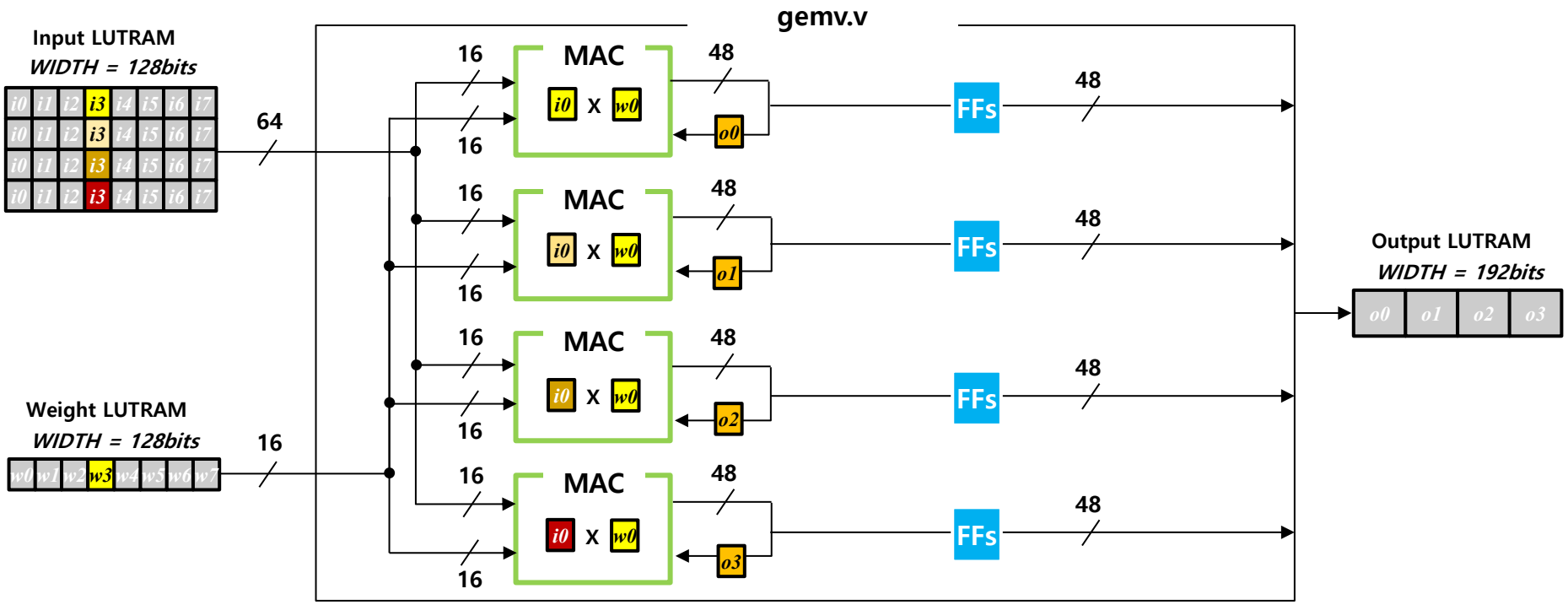
Prob. 3: GEMV Operation with LUTRAM

Output Stationary



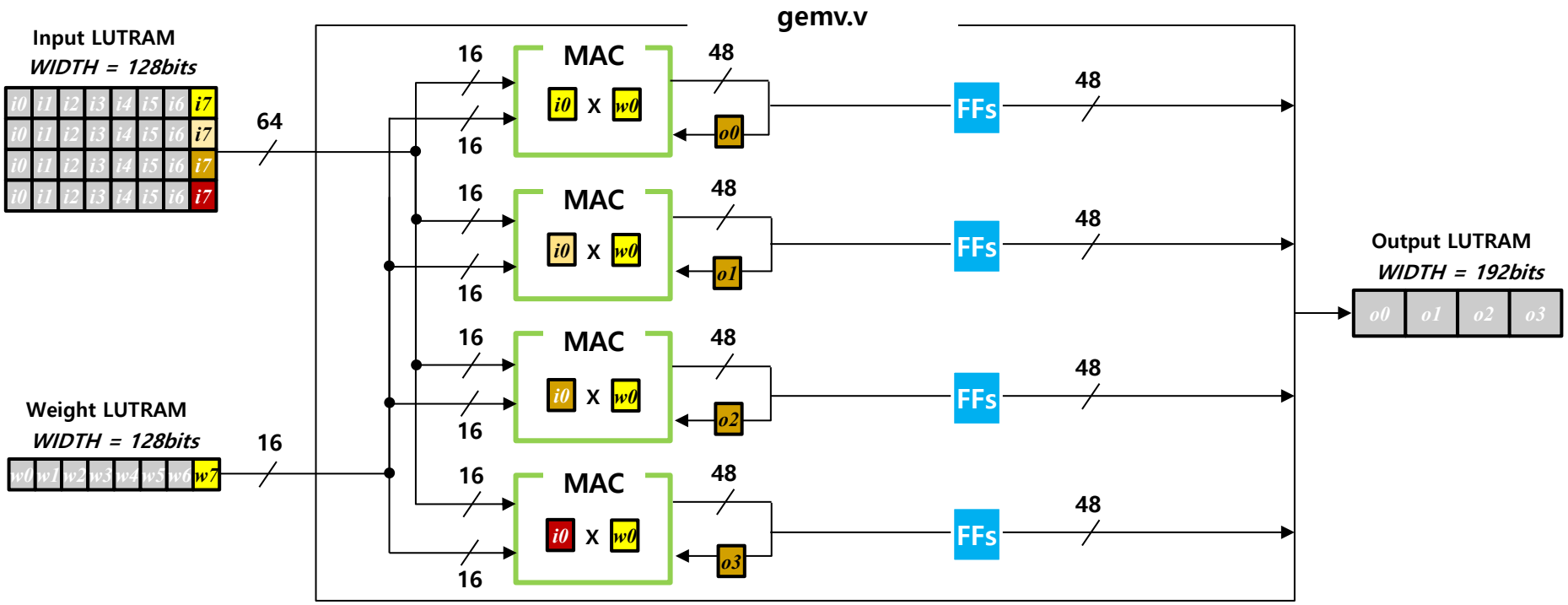
Prob. 3: GEMV Operation with LUTRAM

Output Stationary



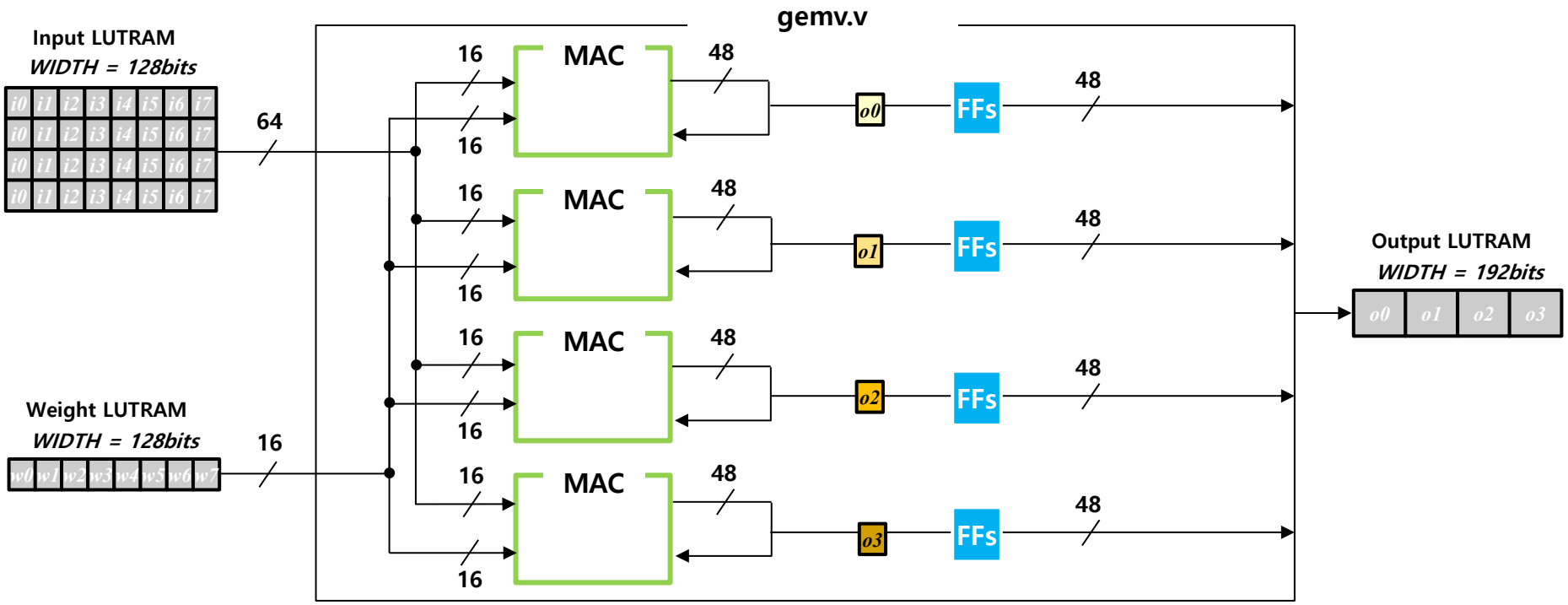
Prob. 3: GEMV Operation with LUTRAM

Output Stationary



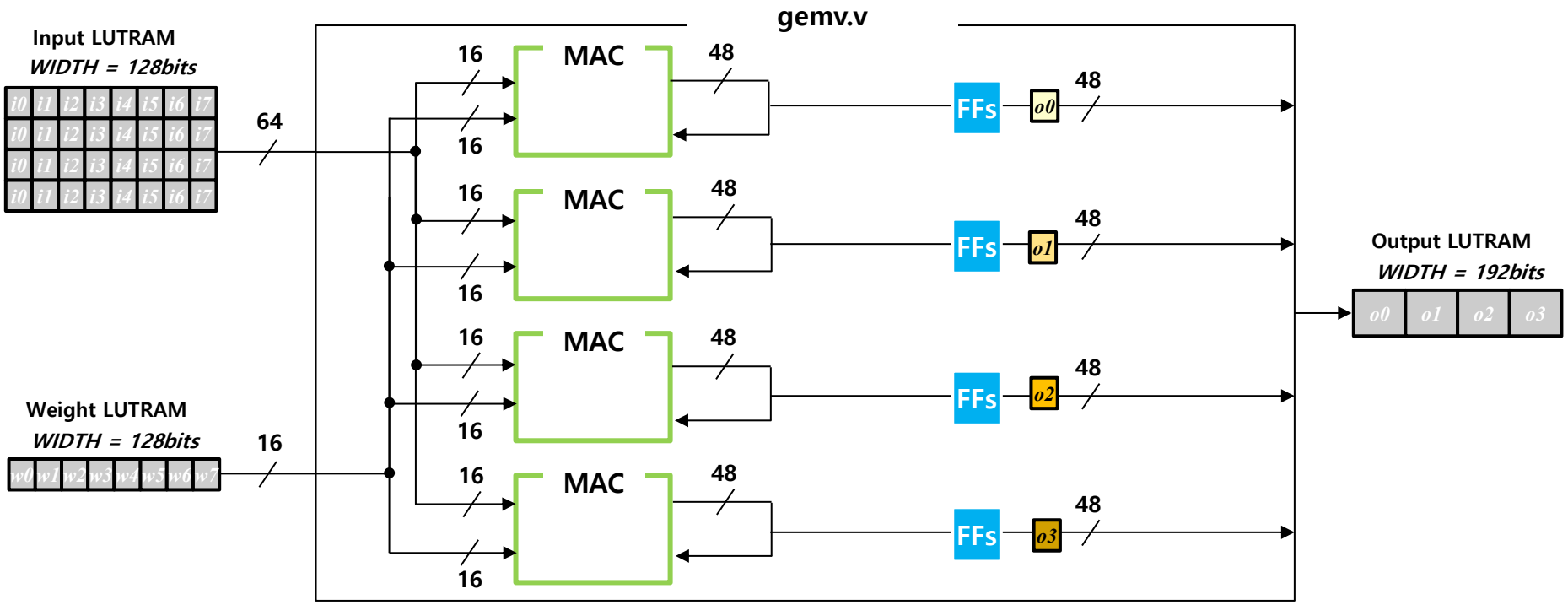
Prob. 3: GEMV Operation with LUTRAM

Output Stationary



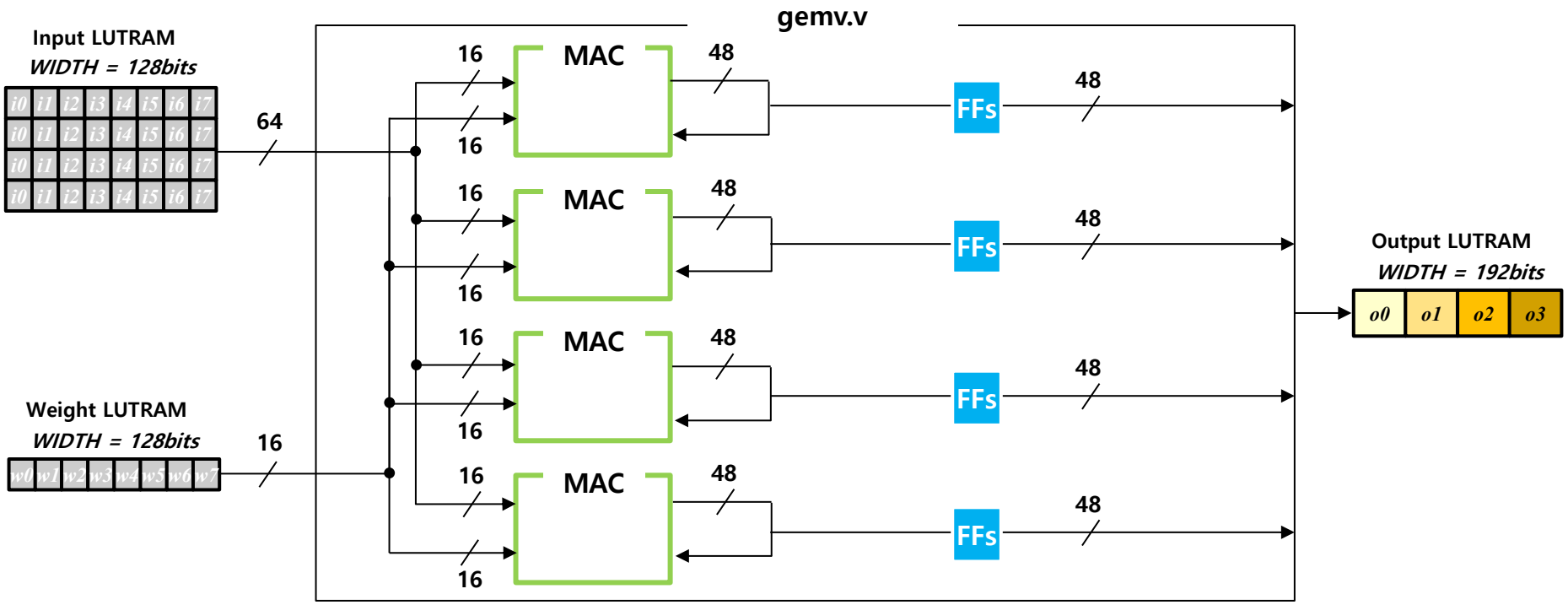
Prob. 3: GEMV Operation with LUTRAM

Output Stationary



Prob. 3: GEMV Operation with LUTRAM

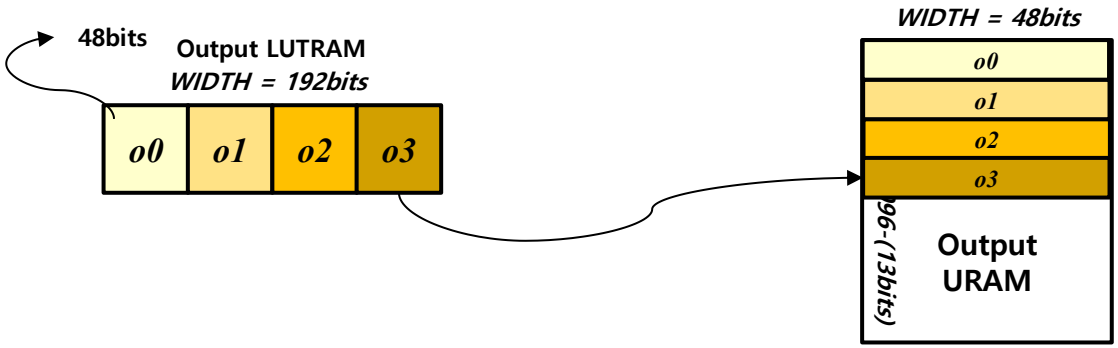
Output Stationary



Prob. 3: GEMV Operation with LUTRAM

Output Stationary

- Cycle 3



Prob. 3: GEMV Operation with LUTRAM

Parameters for lutram_line_buffer_gemv.v

```
parameter INPUT_WIDTH              = ,
parameter WEIGHT_WIDTH             = ,
parameter OUTPUT_WIDTH             = ,
parameter INPUT_DEPTH              = ,
parameter WEIGHT_DEPTH             = ,
parameter OUTPUT_DEPTH             = ,
parameter INPUT_ADDR_WIDTH         = $clog2(INPUT_DEPTH),
parameter WEIGHT_ADDR_WIDTH        = $clog2(WEIGHT_DEPTH),
parameter OUTPUT_ADDR_WIDTH        = $clog2(OUTPUT_DEPTH),
parameter INPUT_LUTRAM_DEPTH       = ,
parameter WEIGHT_LUTRAM_DEPTH      = ,
parameter OUTPUT_LUTRAM_DEPTH      = ,
parameter INPUT_LUTRAM_ADDR_WIDTH  = $clog2(INPUT_LUTRAM_DEPTH),
parameter WEIGHT_LUTRAM_ADDR_WIDTH = $clog2(WEIGHT_LUTRAM_DEPTH),
parameter OUTPUT_LUTRAM_ADDR_WIDTH = $clog2(OUTPUT_LUTRAM_DEPTH),
parameter INPUT_LUTRAM_WIDTH       = ,
parameter WEIGHT_LUTRAM_WIDTH      = ,
parameter OUTPUT_LUTRAM_WIDTH      = ,
parameter INPUT_LUTRAM_RD_WIDTH    = ,
parameter WEIGHT_LUTRAM_RD_WIDTH   = ,
parameter OUTPUT_LUTRAM_RD_WIDTH   = ,
parameter INPUT_LUTRAM_SEL_WIDTH   = $clog2(INPUT_LUTRAM_WIDTH / INPUT_LUTRAM_RD_WIDTH),
parameter WEIGHT_LUTRAM_SEL_WIDTH  = $clog2(WEIGHT_LUTRAM_WIDTH / WEIGHT_LUTRAM_RD_WIDTH),
parameter OUTPUT_LUTRAM_SEL_WIDTH  = $clog2(OUTPUT_LUTRAM_WIDTH / OUTPUT_LUTRAM_RD_WIDTH)
```

Prob. 3: GEMV Operation with LUTRAM

MAC.v

```
`timescale 1ns / 1ps

module MAC(
    input wire i_clk,
    input wire i_rstn,

    input wire i_dsp_enable,
    input wire signed [15:0] i_dsp_input,
    input wire signed [15:0] i_dsp_weight,

    output reg signed [47:0] o_dsp_output
    );

    wire signed [47:0] o_dsp_output_reg;

    always @(posedge i_clk) begin
        if (~i_rstn) begin
            o_dsp_output <= 48'sd0;
        end
        else if (i_dsp_enable) begin
            o_dsp_output <= o_dsp_output_reg;
        end
    end

    dsp_macro_0 dsp (
        .CLK(i_clk),                // input wire CLK
        .CE(i_dsp_enable),          // input wire CE
        .A(i_dsp_input),            // input wire [15 : 0] A
        .B(i_dsp_weight),           // input wire [15 : 0] B
        .P(o_dsp_output_reg) // output wire [47 : 0] P
    );

endmodule
```

INSTANTIATION for MAC.v

```
MAC your_instance_name (
        .i_clk        (),
        .i_rstn       (),
        .i_dsp_enable(),
        .i_dsp_input(),
        .i_dsp_weight(),
        .o_dsp_output()
    );
```

Prob. 3: GEMV Operation with LUTRAM

simple_line_lutram.v

```
`timescale 1ns / 1ps

module simple_line_lutram #(
    parameter WIDTH          = 128,
    parameter DEPTH          = 4,
    parameter ADDR_WIDTH     = $clog2(DEPTH),
    parameter RD_WIDTH       = 16,
    parameter BANK_DEPTH     = WIDTH / RD_WIDTH,
    parameter BANK_SEL_WIDTH = $clog2(BANK_DEPTH)
)(
    input  wire                    clk,
    input  wire                    wr_en,
    input  wire [ADDR_WIDTH-1:0]     wr_addr,
    input  wire [WIDTH-1:0]          wr_din,
    input  wire                    rd_en,
    input  wire [ADDR_WIDTH-1:0]     rd_addr,
    input  wire [BANK_SEL_WIDTH-1:0] rd_sel,
    output reg  [RD_WIDTH-1:0]       rd_dout
);

    wire [BANK_DEPTH*RD_WIDTH-1:0] bank_do;  // data from all banks
    wire [RD_WIDTH-1:0]            sel_do;   // selected bank output
```

```
genvar b;
    generate
        for (b = 0; b < BANK_DEPTH; b = b + 1) begin : gen_bank
            integer i;
            (* ram_style = "distributed" *) reg [RD_WIDTH-1:0] ram
[0:DEPTH-1]; // one LUTRAM bank

            initial begin
                for (i = 0; i < DEPTH; i = i + 1)
                    ram[i] = {RD_WIDTH{1'b0}};
            end

            always @(posedge clk) begin
                if (wr_en)
                    ram[wr_addr] <= wr_din[(b*RD_WIDTH) +: RD_WIDTH];
            end

            assign bank_do[(b*RD_WIDTH) +: RD_WIDTH] = ram[rd_addr];
        end
    endgenerate

    assign sel_do = bank_do[(rd_sel*RD_WIDTH) +: RD_WIDTH];

    always @(posedge clk) begin
        if (rd_en)
            rd_dout <= sel_do;
        else
            rd_dout <= {RD_WIDTH{1'b0}};
    end
endmodule
```

Prob. 3: GEMV Operation with LUTRAM

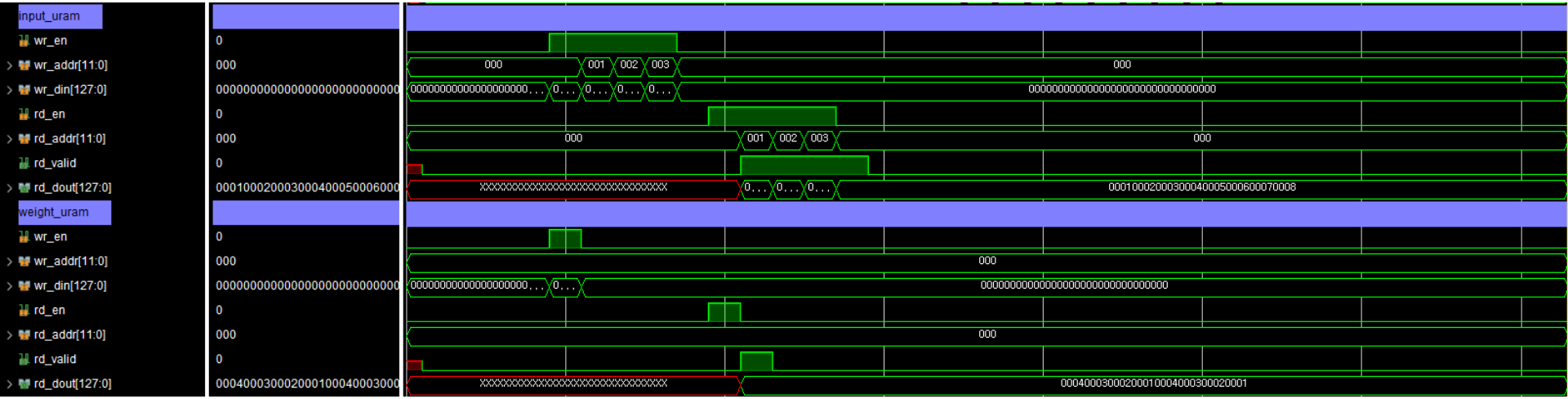
INSTANTIATION for simple_line_lutram.v

```
simple_line_lutram #(
        .WIDTH          (),
        .DEPTH          (),
        .ADDR_WIDTH     (),
        .RD_WIDTH       (),
        .BANK_SEL_WIDTH()
    ) u_wt_lut (
        .clk    (),
        .wr_en  (),
        .wr_addr(),
        .wr_din (),
        .rd_en  (),
        .rd_addr(),
        .rd_sel (),
        .rd_dout()
    );
```

Prob. 3: Waveform (1)

Waveform

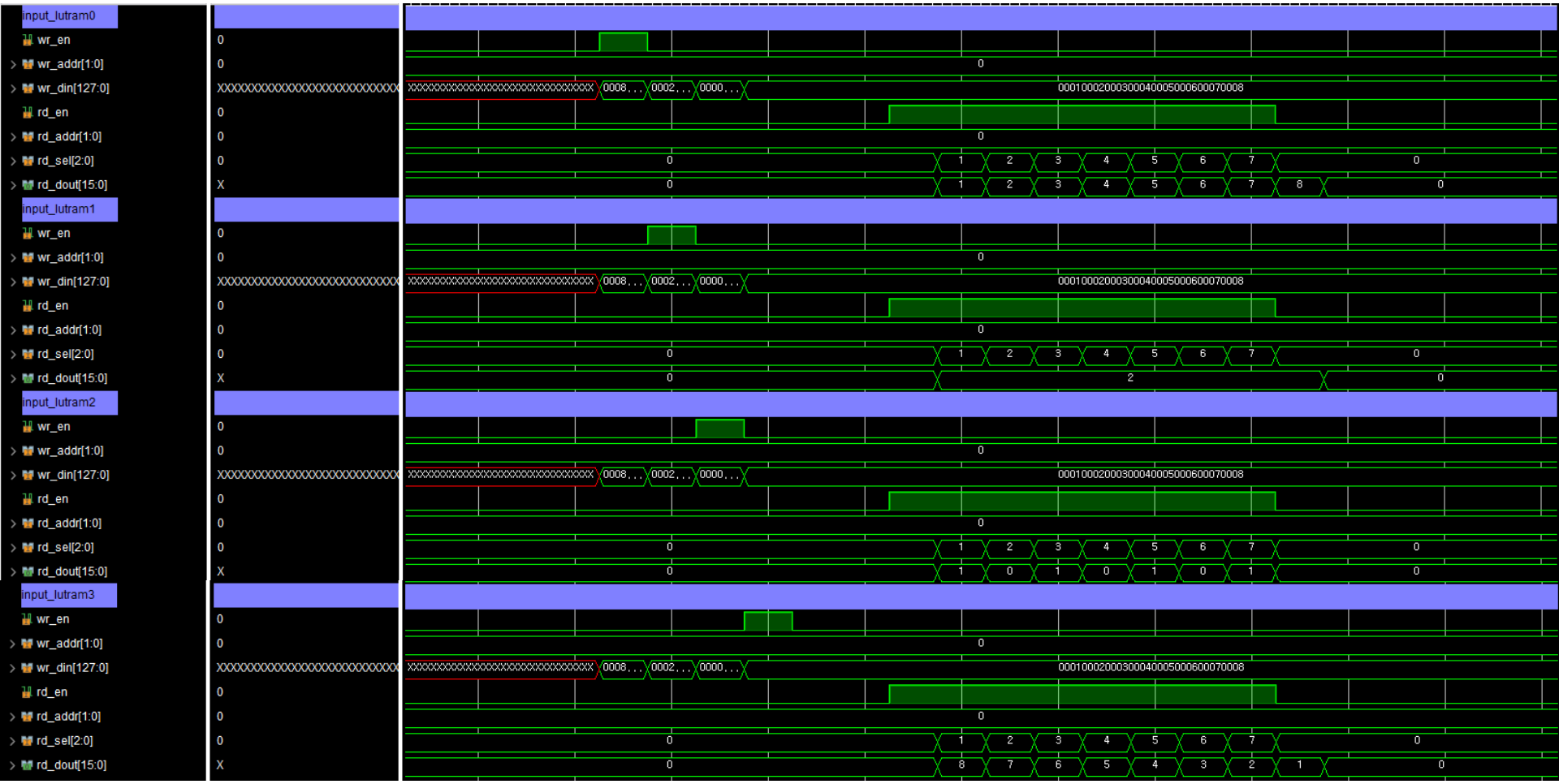
- Input URAM, Weight URAM



Prob. 3: Waveform (2)

Waveform

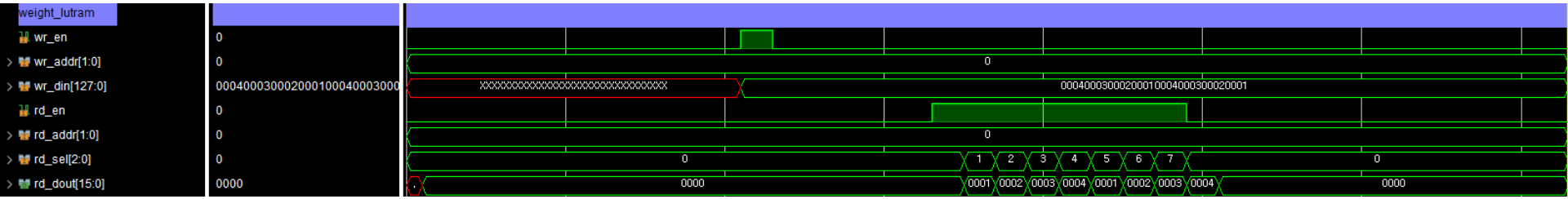
- Input LUTRAM (inputs of GEMV)



Prob. 3: Waveform (3)

Waveform

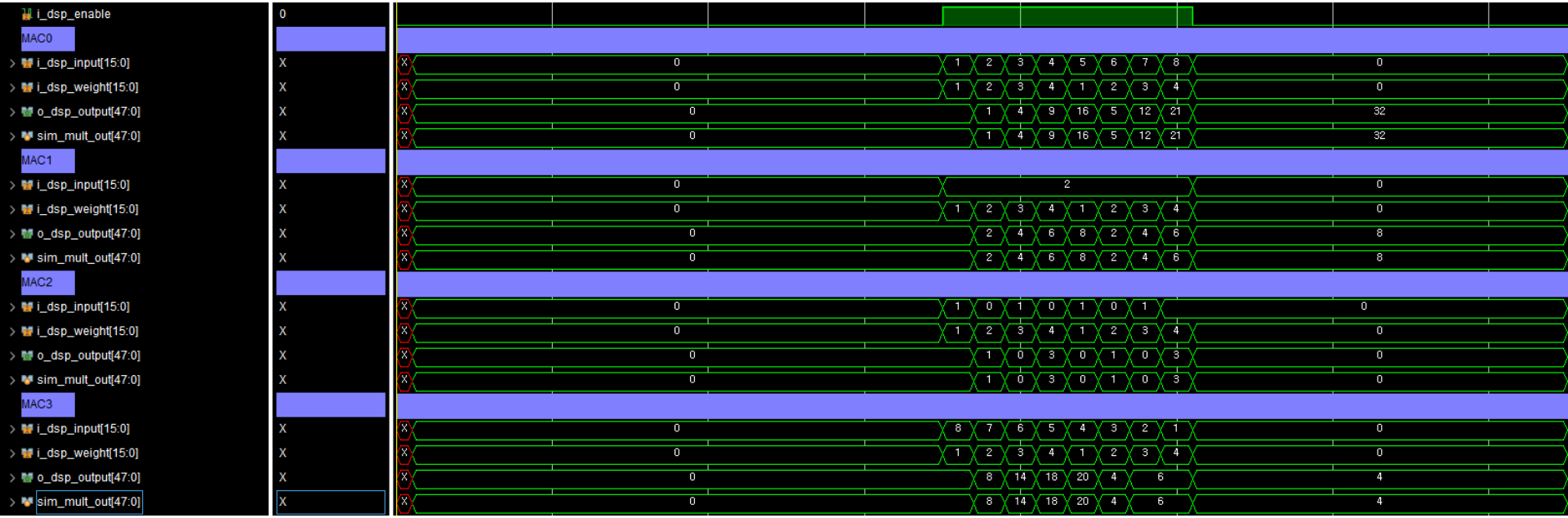
- Weight LUTRAM (inputs of GEMV)



Prob. 3: Waveform (4)

Waveform

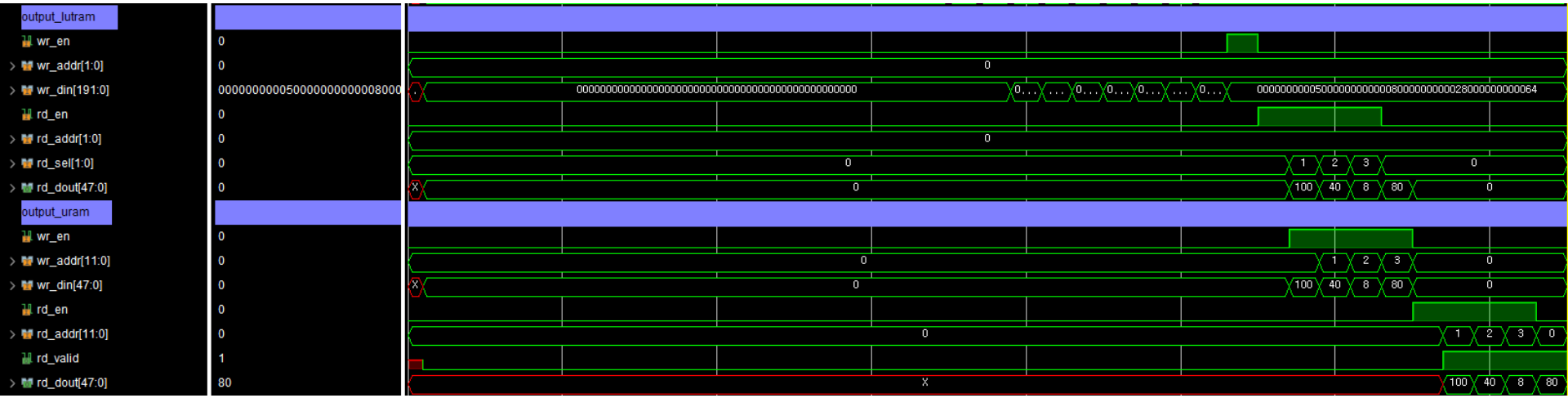
- GEMV operation



Prob. 3: Waveform (5)

Waveform

- Output LUTRAM, Output URAM (outputs of GEMV)



Contents

- Ultra Random Access Memory (URAM)
- Problem 1: URAM Write And Read
- GEMV Operation
- Problem 2: GEMV Operation with URAM
- Problem 3: GEMV Operation with LUTRAM
- Appendix: URAM Tiling and Usage

URAM Tiling and Usage (1)

URAM Usage in Tiled Memory Mapping

- URAM resources **cannot** be flexibly **split or combined**.
- 1 URAM = **one** URAM288 = 4096 × 72
  - Bit-width: 72
  - Depth: 4096

Name	Constraints	Status	URAM
> ✓ **synth_1** (active)	**constrs_1**	**synth_design Complete!**	**1**

- 2 URAM = **two** URAM288 = 4096 × 73
  - Bit-width: 73
  - Depth: 4096

Name	Constraints	Status	URAM
> ✓ **synth_1** (active)	**constrs_1**	**synth_design Complete!**	**2**

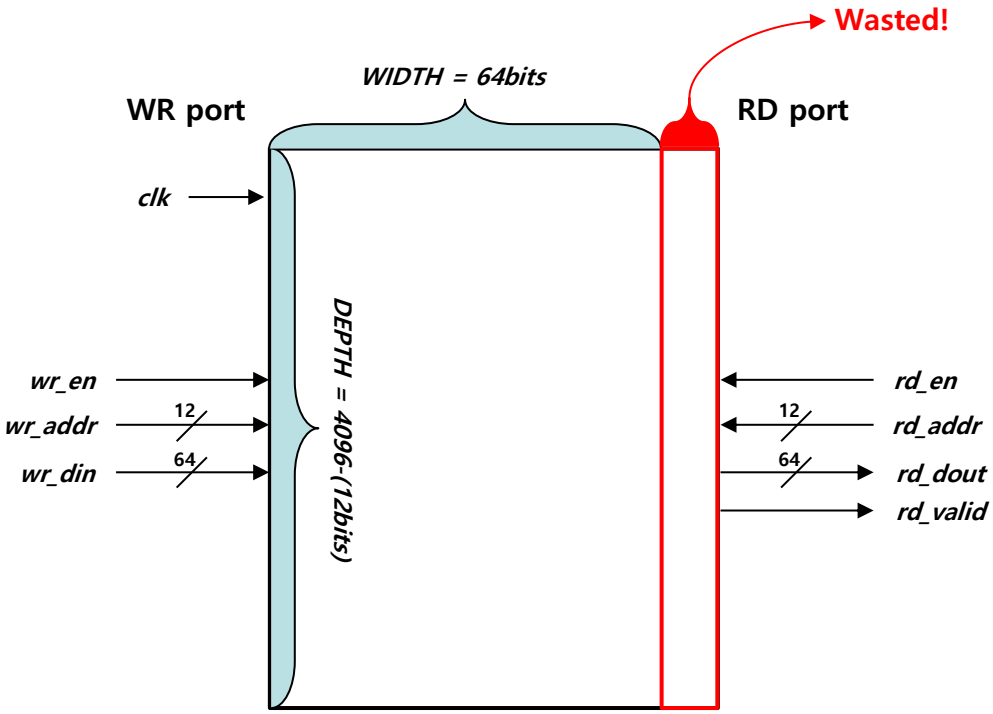
- 2 URAM = **two** URAM288 = 2048 × 73
  - Bit-width: 73
  - Depth: 2048

Name	Constraints	Status	URAM
> ✓ **synth_1** (active)	**constrs_1**	**synth_design Complete!**	**2**

URAM Tiling and Usage (2)

URAM Usage in Tiled Memory Mapping

- URAM resources **cannot** be flexibly **split or combined**.
- **One** URAM288 = 4096 × 64
  - Bit-width: 64
  - Depth: 4096

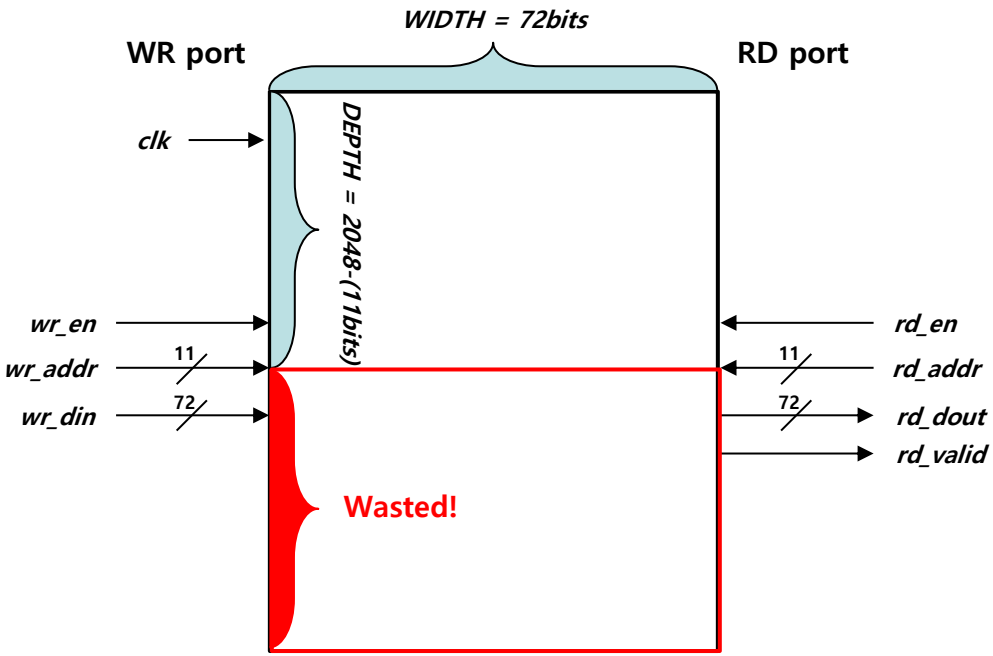


< Simple Dual Port URAM >

URAM Tiling and Usage (3)

URAM Usage in Tiled Memory Mapping

- URAM resources **cannot** be flexibly **split or combined**.
- **One** URAM288 = 2048 × 72
  - Bit-width: 72
  - Depth: 2048

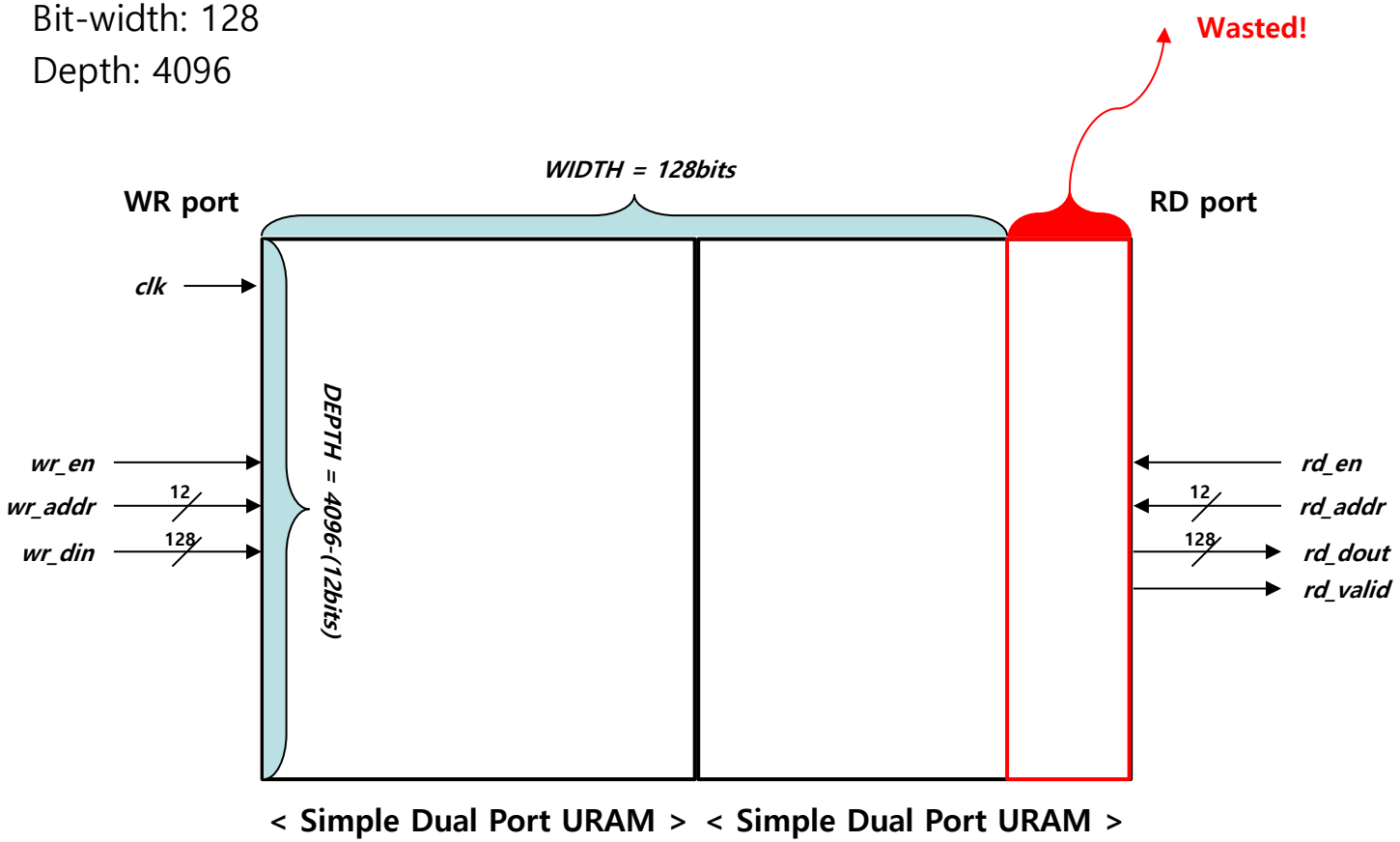


< Simple Dual Port URAM >

URAM Tiling and Usage (4)

URAM Usage in Tiled Memory Mapping

- URAM resources **cannot** be flexibly **split or combined**.
- **Two** URAM288 = 4096 × 128
  - Bit-width: 128
  - Depth: 4096



Thank You!